EUROPEAN AND AMERICAN OPTION PRICING: BINOMIAL TREES, MONTE CARLO AND THE VARIATIONAL PDE

BOLONG TANG

Abstract. We compare discrete-time and continuous-time models for pricing and hedging options on two problems: a one-year at-the-money European call on NVIDIA and an American put. A scaled symmetric random walk converges to Brownian motion, with a Kolmogorov distance at the terminal time that falls like $N^{-1/2}$ with the exact constant 0.399. Path functionals such as the running maximum converge at the same slow rate, which is why discretely monitored barriers need a correction. The CRR tree reaches the Black–Scholes price of \$38.7493 to within \$0.0009 at $N = 10{,}000$ steps, whereas the Euler–Maruyama Monte Carlo price has a standard error of \$0.096 at $5 \cdot 10^5$ paths. A Leisen–Reimer tree with only 101 steps is accurate to \$0.0002, and a control variate reduces the Monte Carlo variance by a factor of 7.7. For the American put, a CRR tree, a finite-difference solver for the variational inequality and least-squares Monte Carlo agree on a value of about 0.71108 for the baseline contract, and the early-exercise premium is 2.5% of the European put at the money. On the NVIDIA inputs the American put is worth \$32.5674, 2.03% more than the European put. Daily delta hedging of the call leaves a hedging error with a standard deviation of 4.9% of the premium, and a wrong volatility distorts the American exercise rule asymmetrically: exercising too early costs about twenty times as much as exercising too late in our worst cases.

1. Introduction

Option prices are needed continuously, for thousands of contracts at a time, both to quote them and to measure the risk of a book. A closed-form price exists only for the simplest contracts, so in practice trees, Monte Carlo simulation and PDE solvers do most of the work, and each of them trades accuracy against computing time. These methods come from two modelling languages: a discrete-time language, in which the stock follows a random walk on a lattice, and a continuous-time language, in which it follows a geometric Brownian motion. The two languages describe the same market in the limit of small time steps, so the useful question is not whether they agree but how quickly, and at what computational price. We study two contracts that stress the methods in different ways. The European call on NVIDIA has a closed-form Black–Scholes price and is therefore a clean benchmark, whereas the American put has no closed form because its holder may exercise at any time and the optimal exercise boundary has to be found together with the price. The question we answer is a practical one: which method should be used for which job, given measured errors and measured run times?

Section 3 shows that a scaled symmetric random walk converges to Brownian motion and measures the convergence in three ways: quadratic and total variation, an exact Kolmogorov distance at the terminal time, and the law of the running maximum. This is the bridge between the discrete and the continuous language, because the binomial tree is a random walk with drift. Section 4 prices a one-year at-the-money call on NVIDIA with the Cox–Ross–Rubinstein (CRR) tree and with Monte Carlo simulation of geometric Brownian motion, and benchmarks both against Black–Scholes. It then asks what a practitioner would ask next: how to make each method more accurate, how to compute sensitivities, how to read the price as an implied volatility, and how

well the option can be hedged in discrete time. Section 5 turns to the American put: the optimal stopping problem and its variational PDE, the closed-form perpetual put, the CRR tree with its exercise boundary and hedge, and independent checks by a finite-difference solver, least-squares Monte Carlo and a quasi-analytic approximation. It closes with the early-exercise premium, dividends, an American put on the NVIDIA inputs, and the profit and loss of the buyer under the pricing measure and under the real-world measure. Section 6 compares every method on accuracy and run time, and Section 7 states the conclusions and their limits.

The classes that were written by hand (`BrownianMotion`, `symmetric_random_walk`, `CRRBinomialModel`, `CRRTree` and the two GBM simulators) are used exactly as written. Every extension is a subclass or a new function placed after them and marked `[EXT]` in the notebook, so each added result can be traced back to the original code it builds on. A single notebook produces every figure, table and number in this report, all random numbers are seeded, and the numbers quoted in the text are read from a file generated by the notebook. A complete run takes about ten minutes on one core. The scope is deliberately narrow: constant volatility, a constant interest rate and no dividends, except in the one section where a dividend yield is introduced on purpose. These are the assumptions under which every method in the report can be compared against an exact or near-exact answer. Full derivations are collected in Appendix A and the code listing is in Appendix C.

2. Setting and notation

The market has a bond $B_t = e^{rt}$ and a stock whose price follows, under the real-world measure $\mathbb{P}$,

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t, \tag{1}$$

where W is a Brownian motion, μ is the real-world drift and σ the volatility. Under the risk-neutral measure $\mathbb{Q}$ the drift μ is replaced by r, so that $e^{-rt}S_t$ is a martingale, and the arbitrage-free price at time 0 of a European claim paying $h(S_T)$ is $e^{-rT}\mathbb{E}^{\mathbb{Q}}[h(S_T)]$. The drift μ therefore plays no role in pricing; it matters only when we simulate profit and loss under $\mathbb{P}$ in Section 5. We assume a constant volatility and interest rate, no dividends, no transaction costs and trading in continuous time, which are the textbook assumptions and are violated in real markets in ways that Section 7 lists. Dividends are introduced in Section 5.10 and discrete trading in Section 4.9. With one stock driven by one Brownian motion and one bond the market is complete, so $\mathbb{Q}$ is unique and every claim is replicable by a self-financing portfolio. This is what makes the price of an option the cost of its hedge, and it is the principle behind the hedging experiments of Sections 4.9 and 5.7.

The European problem uses the NVIDIA (NVDA) closing price of \$238.90 on 2026-10-05, a strike $K = S_0$ (at the money) and a maturity of one year, which is 252 trading days. Time is measured in trading years, so a tree step of one day is $\Delta t = 1/252$. The volatility is the standard deviation of the last 250 daily log returns multiplied by $\sqrt{252}$, which gives $\sigma = 37.73\%$. Using a historical volatility assumes that the next year looks like the last one; an implied volatility from market quotes would replace it in practice. The risk-free rate is the Bank of Canada one-year Treasury-bill yield, $r = 2.9\%$, chosen to match the one-year maturity. A Canadian rate is not the natural choice for a US-dollar asset, but the call is not very sensitive to it: its rho is about 1.057 dollars per percentage point of the rate (Section 4.7). The American put uses a deliberately simple baseline, $S_0 = K = 10$, $\sigma = 20\%$, $r = 2\%$, $T = 1$ and a real-world drift $\mu = 5\%$, for which the perpetual limit is known exactly. Because the exercise boundary scaled by the strike depends only on σ, r and the time to maturity, results for this baseline carry over to any strike; Table 1 lists both parameter sets.

	European call (Part B)	American put (Part C)
Underlying	NVDA, close of 2026-10-05	generic, unit-free
Spot S_0 and strike K	238.90 and $K = S_0$	10 and $K = S_0$
Volatility σ	37.73% (historical, 252 days)	20% (varied: 10%, 30%)
Interest rate r	2.9% (Bank of Canada, 1 year)	2% (varied: 0%, 4%)
Maturity T	1 year (252 trading days)	1 year (varied up to ∞)
Real-world drift μ	10% (hedging experiments only)	5% (profit-and-loss under $\mathbb{P}$)
Dividend yield q	0	0 (varied in Section 5.10)

TABLE 1. Parameters of the two benchmark problems.

3. Part A: Brownian motion and scaled random walks

A process $W = \{W_t\}_{t\geq 0}$ is a Brownian motion on $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ if it is adapted with $W_0 = 0$, if $W_t - W_s \sim N(0, t-s)$ for $0 \leq s \leq t$, if $W_t - W_s$ is independent of $\mathcal{F}_s$, and if $t \mapsto W_t(\omega)$ is continuous. Independence and the Gaussian law of increments fix all finite-dimensional distributions, and continuity of the paths is what separates Brownian motion from other processes with the same increments. The scaled symmetric random walk replaces the Gaussian increments by coin flips: with $\xi_i = \pm 1$ equally likely,

$$W_{t_k}^{(N)} = \sqrt{T/N} \sum_{i=1}^{k} \xi_i, \qquad t_k = kT/N, \tag{2}$$

so that $\mathbb{E}\, W_{t_k}^{(N)} = 0$ and $\operatorname{Var} W_{t_k}^{(N)} = t_k$ for every N. Mean, variance and independence of increments therefore agree with Brownian motion for every N, whereas the lattice support and the non-Gaussian increments do not. Donsker's theorem states that the whole path $W^{(N)}$ converges in law to W as $N \to \infty$, which is the statement we test numerically. This matters for pricing because the CRR tree of Section 4 is exactly such a walk with a drift, so its convergence to geometric Brownian motion is a consequence of the convergence studied here.

The class `BrownianMotion` builds a path step by step from independent $N(0, \Delta t)$ increments on $[0, 1]$, with $N_1 = 10{,}000$ steps. Five independent paths are drawn for the figure. The subclass `symmetric_random_walk` overrides only the step, replacing the Gaussian draw by $\pm\sqrt{\Delta t}$ with probability one half each. It is run with $N_2 \in \{10, 50, 200, 1000\}$ steps on the same time interval. For the quantitative checks we add vectorised generators that draw 2000 paths per N and compute quadratic and total variation. The Kolmogorov distance and the law of the running maximum are computed exactly from the binomial distribution, so they carry no sampling error.

Figure 1 shows that the walk with $N_2 = 10$ makes very few turns and looks like a polygon, whereas the walk with $N_2 = 1000$ is hard to tell from a Brownian path. Between these two the number of turns and the roughness of the path grow steadily with N_2. The maxima and minima of the walks are comparable for every N_2, because the variance at $t = 1$ is equal to 1 whatever the step size. What changes with N_2 is the fine structure of the path, not its overall range. Neither object has a derivative, so the Brownian paths are not smoother than the walks: it is the walk that becomes Brownian as N_2 grows. This is the point of Donsker's theorem. A picture is suggestive but not a measurement, so the next three paragraphs quantify the convergence.

The quadratic variation $\sum_k (\Delta W_k)^2$ of a Brownian path on $[0, 1]$ is close to 1 and its spread shrinks as N grows; at $N = 10{,}000$ the mean is 1.0000. For the scaled walk it equals 1 exactly, because every squared step is T/N. The total variation $\sum_k |\Delta W_k|$ does not converge: at $N =$

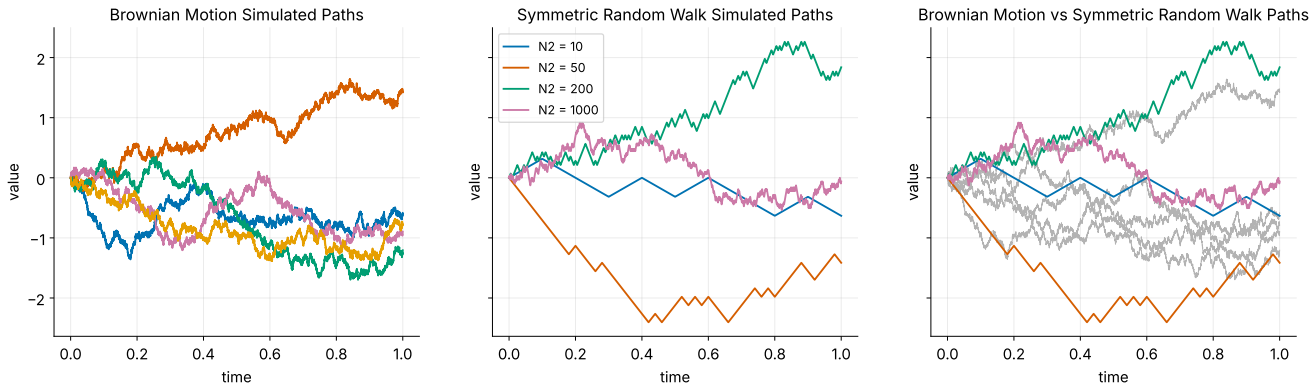


FIGURE 1. Brownian motion paths ($N_1 = 10{,}000$; left), scaled symmetric random walks with $N_2 = 10, 50, 200, 1000$ steps (middle) and both together (right).

10,000 it is 79.8 for the Brownian paths, which matches the theoretical value 79.8 to the digits shown, and it equals $\sqrt{N} = 100$ for the walk. Table 2 shows the same $\sqrt{N}$ growth at every N. Infinite total variation means that $\int \sigma S\, dW$ cannot be defined path by path as an ordinary integral, which is why Itô calculus is needed. Finite and non-zero quadratic variation is what produces the correction $-\frac{1}{2}\sigma^2$ in the drift of $\ln S_t$ in Section 4.2. The same roughness explains why a hedge that is rebalanced only at discrete dates keeps a residual error (Section 4.9).

N	QV Brownian (sd)	QV walk	TV Brownian	TV walk	$\sqrt{2N/\pi}$
10	1.0092 (0.444)	1.0000	2.54	3.16	2.52
50	1.0074 (0.201)	1.0000	5.66	7.07	5.64
200	1.0028 (0.102)	1.0000	11.30	14.14	11.28
1,000	0.9974 (0.044)	1.0000	25.20	31.62	25.23
10,000	1.0000 (0.014)	1.0000	79.79	100.00	79.79

TABLE 2. Quadratic variation (QV) and total variation (TV) of the grid paths on $[0, 1]$, 2000 paths per N. The entry in parentheses is the standard deviation of the QV.

At $t = 1$ the walk equals $(2B - N)/\sqrt{N}$ with $B \sim \mathrm{Bin}(N, \frac{1}{2})$, so its distribution function is a step function that is known exactly. We therefore compute the Kolmogorov distance $\sup_x |F_N(x) - \Phi(x)|$ exactly, checking both sides of every jump, with no sampling error. Table 3 shows that the distance decreases like $N^{-1/2}$: the product of the distance and $\sqrt{N}$ settles at 0.399 as N grows. This constant is $1/\sqrt{2\pi} = \varphi(0)$, half the probability mass of the central lattice cell, which is the size of the jump of the distribution function at its mode. The histograms of Figure 2 show the lattice structure at $N_2 = 10$ and an almost perfect bell shape from $N_2 = 200$ on. The log–log plot in Figure 3 has slope $-\frac{1}{2}$, in agreement with the Berry–Esseen bound. The error in the law of the terminal value is of order $N^{-1/2}$, which is slower than the $O(1/N)$ error of the CRR call price in Section 4, because the price averages a Lipschitz payoff over the lattice.

For a Brownian motion the reflection principle gives $\mathbb{P}(\max_{s \le 1} W_s \ge a) = 2\,(1 - \Phi(a))$. For the simple symmetric walk the same argument works exactly on the lattice and gives $\mathbb{P}(\max_{n \le N} S_n \ge k) = \mathbb{P}(S_N \ge k) + \mathbb{P}(S_N > k)$. Figure 4 shows that the survival function of the walk's maximum lies below the Brownian one, with a largest gap of 0.238 at $N_2 = 10$ and 0.049 at $N_2 = 200$.

N_2	$\sup_x \lvert F_{N_2}(x) - \Phi(x)\rvert$	$\times\sqrt{N_2}$
10	0.12305	0.389
50	0.05614	0.397
200	0.02817	0.398
1,000	0.01261	0.399
10,000	0.00399	0.399
100,000	0.00126	0.399

TABLE 3. Exact Kolmogorov distance between the law of the scaled walk at $t = 1$ and $N(0, 1)$.

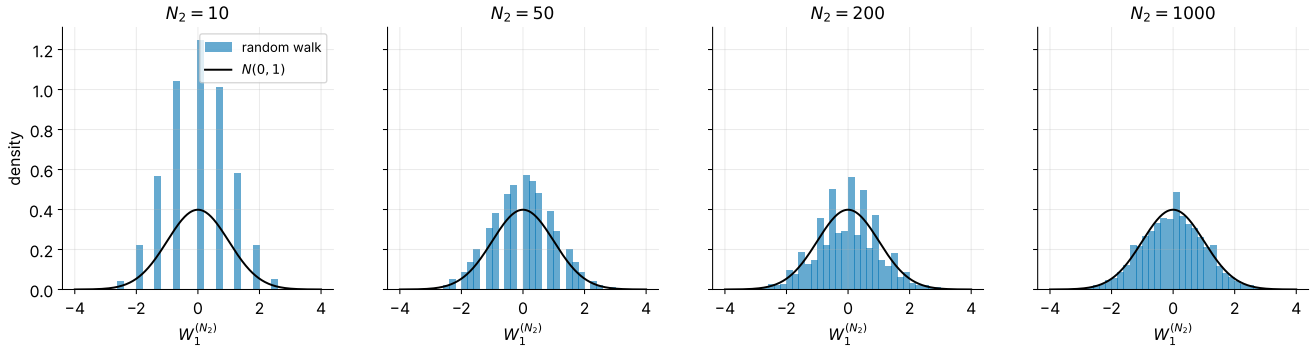


FIGURE 2. Histogram of $W_1^{(N_2)}$ ($2 \cdot 10^4$ walks) against the $N(0, 1)$ density.

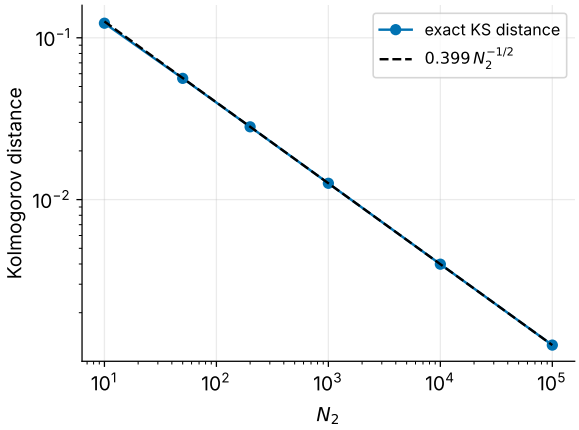


FIGURE 3. Kolmogorov distance against N_2 (log–log) with the limiting rate $0.399\,N_2^{-1/2}$.

The gap shrinks only like $N_2^{-1/2}$, the same slow rate as for the terminal law. A discrete walk can cross a level only at a monitoring date, so it misses the excursions that happen between dates and its maximum is biased downwards. The same bias appears whenever a path-dependent payoff is computed on a lattice or on a simulated grid. For a barrier option this means that a tree or a simulation overstates the value of a knock-out and understates that of a knock-in unless the barrier is shifted by an amount of order $0.58\,\sigma\sqrt{\Delta t}$.

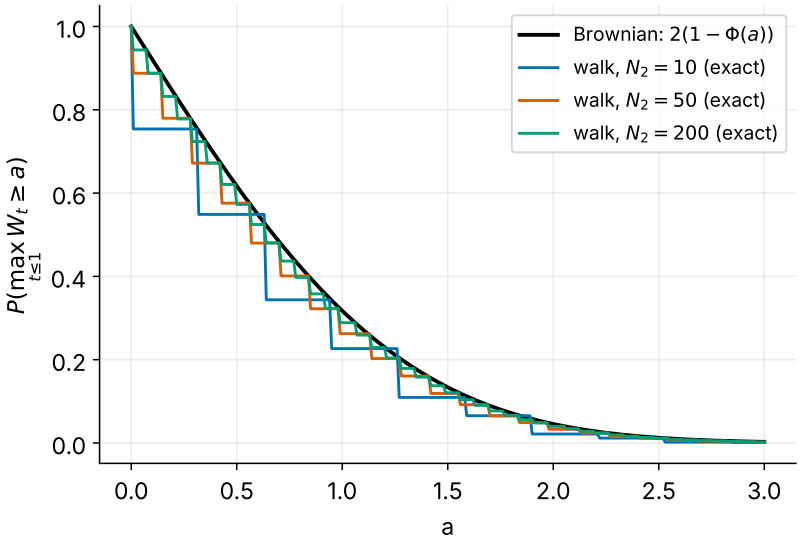


FIGURE 4. Survival function of the running maximum: exact for the walk (three values of N_2) and Brownian (reflection principle).

4. Part B: a European call on NVIDIA

4.1. **The CRR model.** Divide $[0, T]$ into N steps of length $\Delta t = T/N$ and let the stock move in each step to uS or dS with $u = e^{\sigma\sqrt{\Delta t}}$ and $d = 1/u$, so that after j steps with i up-moves the price is $S_{j,i} = S_0 u^{2i-j}$. Because $ud = 1$ the tree recombines, layer j has only $j + 1$ nodes, and the choice of u makes the variance of the one-step log-return equal to $\sigma^2\Delta t$. No arbitrage forces the discounted stock to be a martingale under the pricing probability p of an up-move, $S = e^{-r\Delta t}\big(p\,uS + (1-p)\,dS\big)$, which is solved by

$$p = \frac{e^{r\Delta t} - d}{u - d}, \qquad 0 < p < 1 \iff d < e^{r\Delta t} < u, \tag{3}$$

and the second condition holds for all small enough Δt. To see why this is also a hedging price, choose a number of shares Δ and a bond holding B with $\Delta uS + Be^{r\Delta t} = V_u$ and $\Delta dS + Be^{r\Delta t} = V_d$; subtracting gives $\Delta = \frac{V_u - V_d}{(u-d)S}$. Substituting p shows that the cost $\Delta S + B$ of this portfolio equals $e^{-r\Delta t}\big(pV_u + (1-p)V_d\big)$, so the price and the hedge are the same calculation. The option tree is filled backward from the payoff $V_{N,i} = (S_{N,i} - K)^+$ with $V_{j,i} = e^{-r\Delta t}\big(pV_{j+1,i+1} + (1-p)V_{j+1,i}\big)$, and the price is $V_{0,0}$. The class `CRRBinomialModel` stores every node, which costs $O(N^2)$ time and memory and is ideal for drawing trees and sample paths, whereas the vectorised pricer added for the experiments produces identical prices with $O(N)$ memory.

The terminal log-price of the tree is $\ln S_0 + \sigma\sqrt{\Delta t}\,(2B - N)$ with $B \sim \mathrm{Bin}(N, p)$, a sum of N independent steps. By the central limit theorem it converges to a normal law with mean $\ln S_0 + (r - \frac{1}{2}\sigma^2)T$ and variance $\sigma^2 T$, which is the law of $\ln S_T$ under geometric Brownian motion, so the tree price converges to the Black–Scholes price. The error of the tree price is of order $1/N$, but it is not monotone in N. With $K = S_0$ a terminal node sits exactly at the strike when N is even and the kink falls half-way between two nodes when N is odd, so the error behaves like $(-1)^N c/N$ rather than c/N. Four standard remedies are compared in Section 4.5: averaging the prices for N and $N + 1$ steps; Broadie–Detemple smoothing, which replaces the last layer by the

Black–Scholes value of the one-step claim; Richardson extrapolation $2V_{2N} - V_N$; and the Leisen–Reimer tree, which is built so that the strike coincides with a node boundary. The first three remove the oscillation and the Leisen–Reimer tree additionally raises the order of convergence from one to two. Accelerations that use a closed form at the last step apply only to European payoffs, so for the American put of Part C we keep the plain CRR lattice and control the error by refinement and extrapolation.

4.2. **Geometric Brownian motion and its simulation.** Applying Itô's formula to $f(S) = \ln S$, with $f' = 1/S$ and $f'' = -1/S^2$, gives $d\ln S_t = \frac{dS_t}{S_t} - \frac{1}{2}\frac{(dS_t)^2}{S_t^2} = (\mu - \frac{1}{2}\sigma^2)\,dt + \sigma\,dW_t$, hence

$$S_t = S_0 \exp\Big((\mu - \tfrac{1}{2}\sigma^2)t + \sigma W_t\Big). \tag{4}$$

Hence S_t is lognormal, strictly positive, with $\mathbb{E}[S_t] = S_0 e^{\mu t}$. Under $\mathbb{Q}$ the drift μ is replaced by r, and the call is priced as $e^{-rT}\,\mathbb{E}[(S_T - K)^+]$, estimated by a Monte Carlo average over simulated terminal values. The Euler–Maruyama scheme updates $S_{n+1} = S_n + \mu S_n \Delta t + \sigma S_n \Delta W_n$, which is cheap and generic but can in principle produce negative prices, whereas the exact step $S_{n+1} = S_n \exp\big((\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,Z_n\big)$ keeps the price positive and has no discretisation error. Both are driven by the same random numbers in Figure 8, so any difference between the two panels is discretisation error alone. The functions `gbm_euler_maruyama` and `gbm_exact` return full paths, which is what the figures need. For the price tables we add a version that keeps only the current vector of prices, because $5 \cdot 10^5$ paths with 300 steps would otherwise occupy more than a gigabyte, and we check that it reproduces the original function when both receive the same random numbers.

4.3. **The Black–Scholes benchmark and the Greeks.** Evaluating $e^{-rT}\mathbb{E}^{\mathbb{Q}}[(S_T - K)^+]$ for the lognormal S_T gives

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2), \qquad d_{1,2} = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \tag{5}$$

and the put follows from put–call parity $C - P = S_0 - Ke^{-rT}$. The sensitivities are $\Delta = \Phi(d_1)$, $\Gamma = \varphi(d_1)/(S_0\sigma\sqrt{T})$, $\mathcal{V} = S_0\varphi(d_1)\sqrt{T}$, $\rho = KTe^{-rT}\Phi(d_2)$ and $\Theta = -\frac{S_0\varphi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2)$, where φ is the standard normal density. For the NVIDIA call they take the values $\Delta = 0.6047$, $\Gamma = 0.00427$, vega 92.01 (that is, 0.920 dollars per volatility point), $\Theta = -20.42$ per year and $\rho = 105.71$. Delta is the number of shares that hedges the option, gamma measures how quickly that number changes and therefore how large the hedging error is when rebalancing is discrete, and vega is the exposure to the volatility input, which is the least certain one. Theta is the loss of value from the passage of time alone, about -0.056 dollars per calendar day at the money, and rho the exposure to the interest rate. The price \$38.7493 from Equation (5) is the reference against which every numerical estimate in this part is measured.

4.4. **Results: sample paths and binomial prices.** Figure 7 shows five stock paths and five option-value paths in the tree with $N = 100$, simulated with the risk-neutral probability p. The stock paths fan out like a random walk with drift r, and at this resolution the lattice is no longer visible. The option value follows the stock but is a convex function of it, so it moves faster than the stock when the call is in the money and flattens when it is out of the money. Every path of the option value ends at the payoff $(S_T - K)^+$. Figure 8 compares Euler–Maruyama with the exact solution for the same random numbers and $N = 2000$ steps. The two panels are indistinguishable by eye, and the largest difference between a pair of paths is about a dollar on a price of \$240.

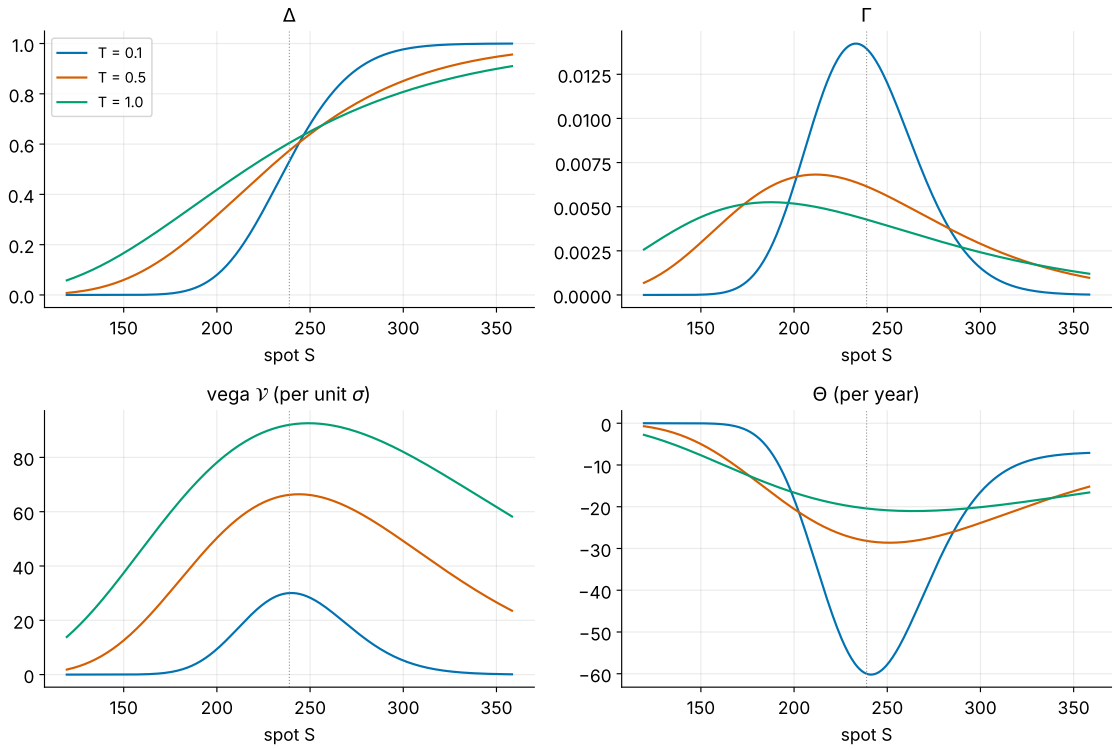


FIGURE 5. Black–Scholes Greeks of the call against spot for three maturities ($K = 238.90$, $r = 2.9\%$, $\sigma = 37.73\%$).

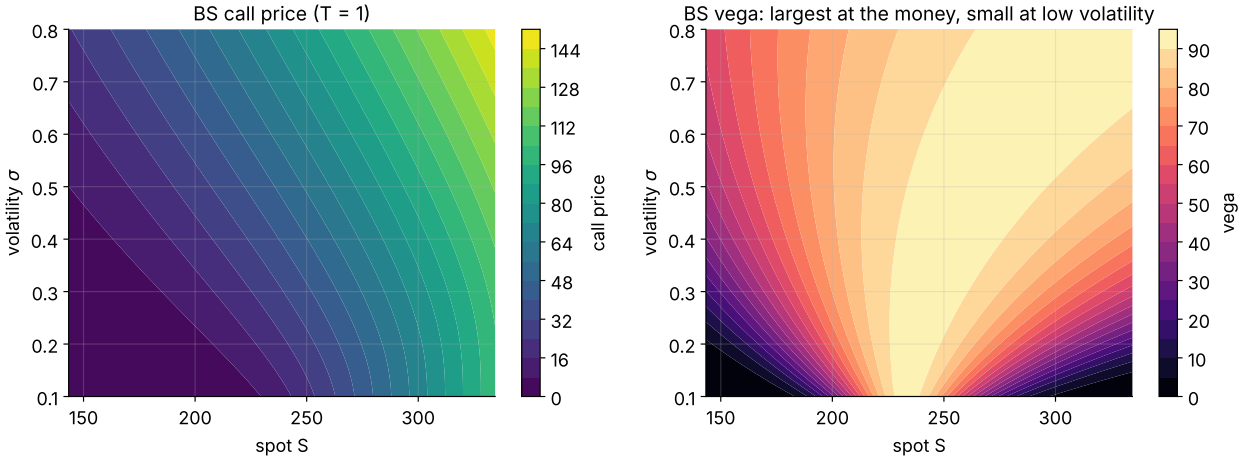


FIGURE 6. Black–Scholes call price (left) and vega (right) as functions of spot and volatility ($T = 1$).

These figures say nothing about the accuracy of prices, which is the subject of the tables that follow.

The CRR price converges to the Black–Scholes price of \$38.7493: at $N = 10{,}000$ it is \$38.7484, an error of \$0.0009. The error is \$0.0882 at $N = 100$ and \$0.0088 at $N = 1000$, so one hundred steps already give the price to within ten cents. The last column of Table 4 shows that N times the absolute error is nearly constant from $N = 50$ on. This is first-order convergence, as predicted

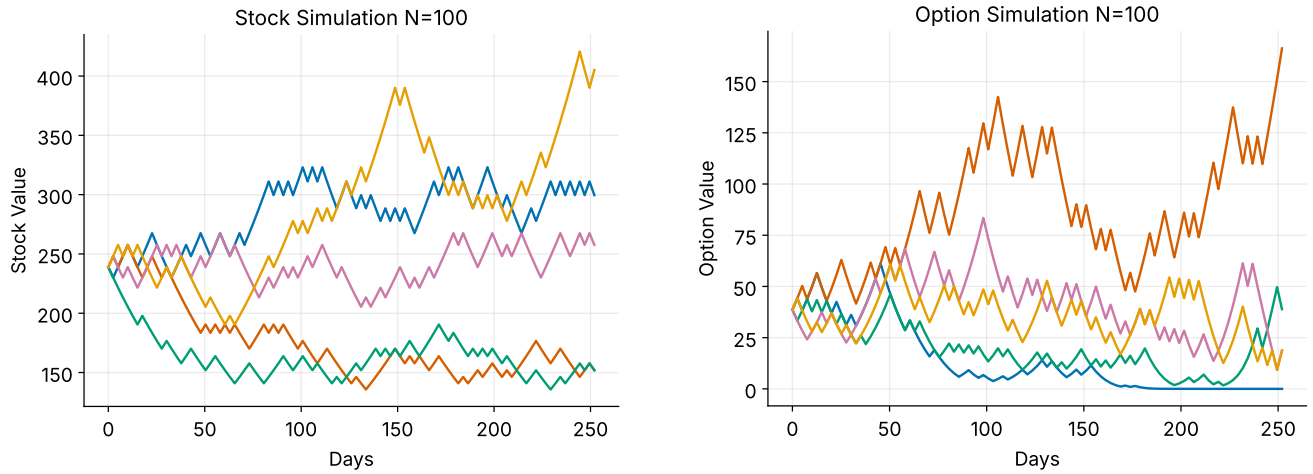


FIGURE 7. Five simulated stock (left) and option (right) value paths in the CRR tree, $N = 100$.

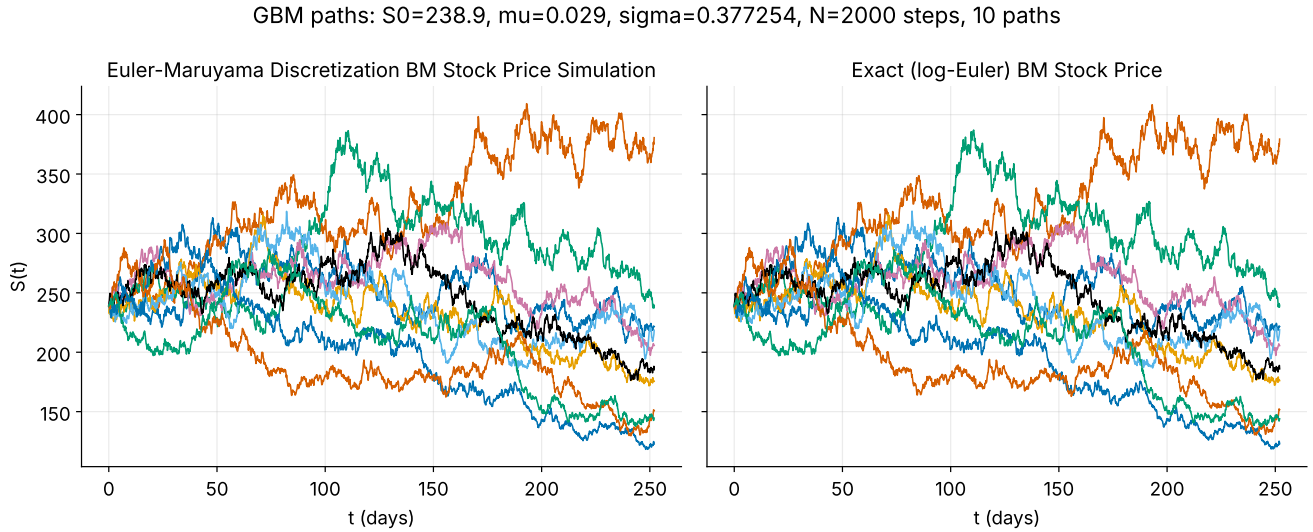


FIGURE 8. GBM paths with Euler–Maruyama (left) and exact (right) simulation, same random numbers.

in Section 4. For small N the error alternates in sign: the tree overprices the call at $N = 5$ and $N = 25$ and underprices it at $N = 10$ and $N = 50$. This is the parity effect of the kink at the strike and is quantified in the next section. The row for $N = 10{,}000$ was computed with the vectorised pricer, since the list-based class would have to store about $5 \cdot 10^7$ nodes.

4.5. **Quality of the binomial price: the saw-tooth and faster trees.** Figure 9 shows that the signed error alternates with the parity of N: N times the error settles at -8.81 for even N and at 8.54 for odd N. In absolute value both sequences decay like $1/N$, so parity changes the sign of the error but not its order. Table 5 compares seven trees at odd N. At $N = 101$ the Leisen–Reimer tree has an error of 1.7×10^{-4} against 8.4×10^{-2} for plain CRR, and at $N = 1001$ its error is 1.7×10^{-6}. On the log–log plot of Figure 10 plain CRR, Jarrow–Rudd and Tian follow slope -1 with visible oscillation, while averaging and smoothing remove most of the oscillation. Only the

N	CRR price	$\lvert\text{CRR} - \text{BS}\rvert$	$N \times \lvert\text{error}\rvert$
5	40.4839	1.7347	8.673
10	37.8785	0.8708	8.708
25	39.0918	0.3425	8.562
50	38.5731	0.1762	8.809
100	38.6611	0.0882	8.820
200	38.7051	0.0441	8.826
500	38.7316	0.0177	8.829
1,000	38.7404	0.0088	8.831
10,000	38.7484	0.0009	8.832

TABLE 4. CRR prices of the one-year call against the Black–Scholes price. $N \leq 1000$ with the list-based class, $N = 10{,}000$ with the vectorised pricer.

Leisen–Reimer tree changes the slope, to -2. For a European payoff with a kink the practical choice is therefore the Leisen–Reimer tree or smoothing combined with Richardson extrapolation. For an American payoff we stay with plain CRR and control the error by refinement, because the accelerations rely on a closed-form value at the last step.

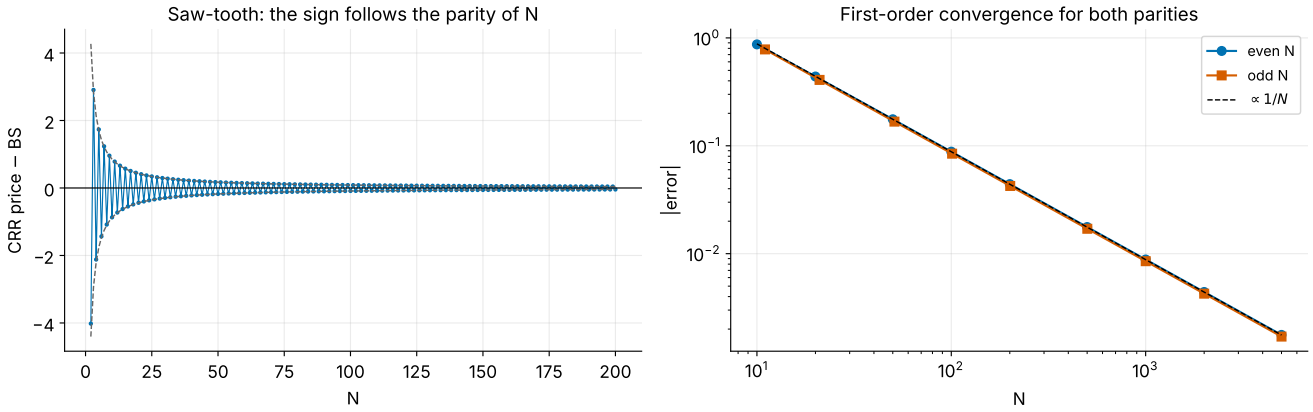


FIGURE 9. Left: signed CRR error for $N = 2, \ldots, 200$ with the envelopes c_{even}/N and c_{odd}/N. Right: absolute error for even and odd N (log–log).

Tree / acceleration	$N = 21$	$N = 101$	$N = 501$	$N = 2001$
CRR	4.1e-01	8.4e-02	1.7e-02	4.3e-03
CRR, average of N and N+1	4.5e-03	9.9e-04	2.9e-04	7.6e-05
CRR + BD smoothing	1.2e-01	2.6e-02	5.4e-03	1.3e-03
CRR + BD smoothing + Richardson	8.6e-03	1.2e-03	2.1e-04	5.1e-05
Jarrow-Rudd	1.5e-01	5.4e-02	7.0e-03	4.7e-03
Tian	3.2e-01	1.3e-02	1.3e-02	2.5e-03
Leisen-Reimer	3.7e-03	1.7e-04	6.9e-06	4.4e-07

TABLE 5. Absolute error of the tree variants for selected odd N.

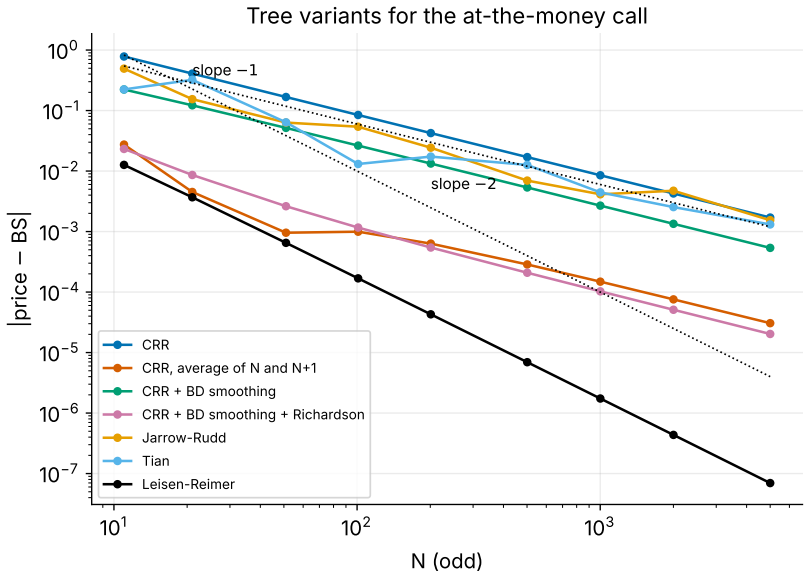


FIGURE 10. Absolute error of seven tree variants at odd N (log–log).

4.6. **Monte Carlo.** With $5 \cdot 10^5$ paths and $N = 300$ steps the Euler–Maruyama Monte Carlo price is \$38.7236 with standard error \$0.096, which is within \$0.0257 of the Black–Scholes price. The other two step counts at the same sample size are closer to the benchmark, so the ordering of the cells in the last row is noise: all three lie within about a third of a standard error of each other. With only 1,000 paths the standard error is above two dollars, about six percent of the price, and the errors in that row are of this size. They are statistical fluctuations, not systematic bias, since they change sign across columns. Increasing the number of time steps from 100 to 300 changes the price by much less than its standard error. The statistical error therefore dominates the Euler bias at these step counts, in line with the weak order of the scheme. Because the standard error falls only like $M^{-1/2}$, a standard error of one cent would need about $5 \cdot 10^9$ paths, whereas the tree reaches that accuracy deterministically with a few thousand steps.

Paths	$N = 100$	$N = 200$	$N = 300$
1,000	41.4717 (2.325)	38.7524 (2.097)	41.2796 (2.218)
10,000	38.5516 (0.681)	39.3903 (0.695)	39.8024 (0.698)
100,000	38.8126 (0.215)	38.6919 (0.216)	38.7307 (0.216)
250,000	38.8783 (0.136)	38.9348 (0.137)	38.6022 (0.136)
500,000	38.7540 (0.096)	38.7559 (0.096)	38.7236 (0.096)

TABLE 6. Euler–Maruyama Monte Carlo prices of the call, price (standard error), for N time steps and the number of paths in the first column.

The strong error $\mathbb{E}|S_T^{EM} - S_T|$ compares each Euler path with the exact path driven by the same noise, whereas the weak error compares the expectation of a payoff and is the only one that matters for pricing. We measure both with common random numbers, so the Monte Carlo noise in the difference is small. The strong error falls from 0.069 at $N = 1$ to 0.0052 at $N = 256$ (in units of S_0), and the log–log slope is -0.49, which is order one half. This is the textbook strong order

of the Euler–Maruyama scheme. The bias of the call price halves each time N doubles between $N = 4$ and $N = 64$, with a log–log slope of -0.96, which is weak order one. At $N = 256$ the bias is $+0.0066$ with a standard error of 0.0022, so it can no longer be separated from zero. For a European call a few hundred steps are therefore enough and the number of paths is the decision that matters; at $N = 1$ the bias even has the opposite sign, which is a pre-asymptotic effect.

N	$\mathbb{E}\lvert S_T^{\mathrm{EM}} - S_T\rvert / S_0$	price bias	(s.e.)
1	0.0692	-0.3169	(0.0323)
2	0.0534	+0.3639	(0.0226)
4	0.0395	+0.3603	(0.0163)
8	0.0286	+0.2208	(0.0119)
16	0.0205	+0.1103	(0.0086)
32	0.0146	+0.0566	(0.0061)
64	0.0103	+0.0255	(0.0044)
128	0.0073	+0.0178	(0.0031)
256	0.0052	+0.0066	(0.0022)

TABLE 7. Strong error and weak error (call-price bias) of Euler–Maruyama, $4 \cdot 10^5$ paths with common random numbers.

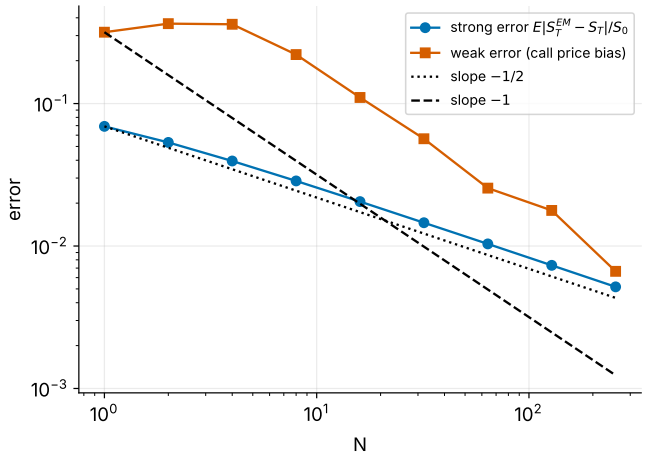


FIGURE 11. Strong and weak error of Euler–Maruyama against N (log–log).

At $M = 262\,144$ samples the antithetic estimator reduces the variance of the plain estimator by a factor of 1.8 and the control variate by a factor of 7.7. Scrambled Sobol quasi-Monte Carlo reduces it by a factor of 76727, which is why it nearly disappears from the log–log plot of Figure 12. The control variate uses the discounted terminal price, whose expectation S_0 is known exactly and which is strongly correlated with the call payoff near the money. The coefficient of the correction is estimated from the same sample, which introduces a negligible bias. Quasi-Monte Carlo is so effective here because the integrand is a function of a single uniform variable, for which low-discrepancy points give an error close to $1/M$. A path-dependent payoff depends on one random number per time step, and the advantage shrinks as this dimension grows unless a Brownian-bridge construction concentrates the variance in the first coordinates. The standard errors are

trustworthy: the 95% intervals of the plain estimator cover the Black–Scholes price in 95.2% of 400 repetitions. Simulating with a real-world drift of 10% instead of r would give a price of \$49.96 instead of \$38.7493, which is the classical error of pricing under the wrong measure.

Method	mean estimate	sd of estimates	RMSE	variance ratio vs plain
plain	38.7455	0.1228	0.1213	1.0
antithetic	38.7763	0.0921	0.0949	1.8
control variate	38.7491	0.0441	0.0436	7.7
Sobol (scrambled)	38.7494	0.0004	0.0005	76726.6

TABLE 8. Variance reduction at $M = 262{,}144$ samples, 40 repetitions. The last column is the variance of the plain estimator divided by that of the method.

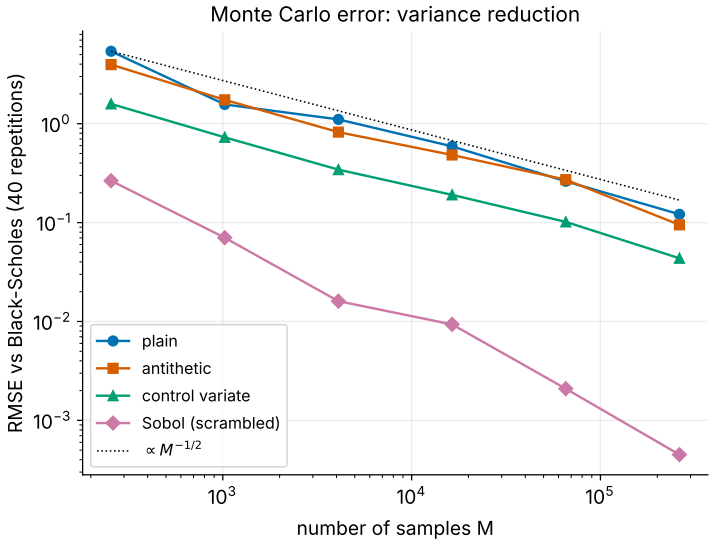


FIGURE 12. RMSE of four Monte Carlo estimators against the number of samples (log–log).

4.7. **Greeks by three routes.** Table 9 shows that all routes agree: Δ is 0.6047 from Black–Scholes, 0.6047 from the tree and 0.6048 from the pathwise Monte Carlo estimator. The tree obtains Δ and Γ for free from its first two layers, while vega and rho need a bump of the inputs. For Γ the tree gives 0.00428 and the Monte Carlo estimator that combines the pathwise and likelihood-ratio ideas gives 0.00428 with standard error 0.00001. A bump-and-revalue estimator with common random numbers has a standard error of 0.00003, which is larger because the second difference divides the noise by the square of the bump size. For vega the pathwise estimator has a smaller standard error than the likelihood-ratio estimator, because it differentiates the payoff, which is Lipschitz, instead of the density. The likelihood-ratio method earns its place when the payoff is too irregular to differentiate, as for the second derivative in Γ. Theta and rho agree between the closed form and the tree, and their size gives a feel for the risks: the call loses about -0.056 dollars per calendar day at the money and gains about 1.057 dollars for a one-point rise in the rate. These are the numbers a desk would use to decide how often to re-price and how sensitive the book is to the input rate.

Greek	Black-Scholes	Tree ($N = 1000$)	MC estimator	MC value	s.e.
Δ	0.60469	0.60466	pathwise	0.60476	0.00070
Δ	0.60469	0.60466	LR	0.60653	0.00163
Δ	0.60469	0.60466	bump (CRN)	0.60474	0.00070
Γ	0.00427	0.00428	LR x pathwise	0.00428	0.00001
Γ	0.00427	0.00428	bump (CRN)	0.00424	0.00003
vega	92.00674	91.98375	pathwise	92.22585	0.22333
vega	92.00674	91.98375	LR	93.04232	0.72616
Θ (per year)	-20.4206	-20.4347	–	–	–
ρ	105.711	105.713	–	–	–

TABLE 9. Delta, gamma and vega of the call from the closed form, from the CRR tree ($N = 1000$) and from Monte Carlo (10^6 samples). The standard error is that of the Monte Carlo value.

4.8. **Implied volatility.** The implied volatility of an observed price is the $\hat{\sigma}$ that solves $C_{BS}(\hat{\sigma}) = C_{obs}$. The root is unique because the Black–Scholes price is strictly increasing in σ, since vega is positive, and we find it with Brent's method. Applied to the tree prices, the implied volatility differs from the true 37.73% by -0.10 volatility points at $N = 100$ (the tree underprices the call), and it inherits the saw-tooth of the price, as Figure 13 shows. At $N = 10{,}000$ the difference is below one hundredth of a volatility point. For the best Monte Carlo price the 95% interval of the price maps to an implied-volatility interval of 37.49% to 37.90%. This is consistent with dividing the standard error of the price by vega, which is the quickest way to judge whether a simulation is accurate enough for a given tolerance in volatility. Expressing numerical error in volatility points makes it directly comparable with the uncertainty in the volatility input itself, which is usually far larger.

Method / N	price	implied vol (%)	error (vol points)
10	37.8785	36.779	-0.946
25	39.0918	38.098	+0.372
50	38.5731	37.534	-0.191
100	38.6611	37.630	-0.096
250	38.7140	37.687	-0.038
1,000	38.7404	37.716	-0.010
10,000	38.7484	37.724	-0.001
MC $5 \cdot 10^5$ (Euler, $N = 300$)	38.7236	37.697	-0.028

TABLE 10. Implied volatility of the numerical prices (true volatility 37.73%).

4.9. **Discrete delta hedging.** The seller of the call receives the Black–Scholes price C_0, holds $\Delta_k = \Phi(d_1)$ shares at n equally spaced dates and finances the position at rate r. The discounted

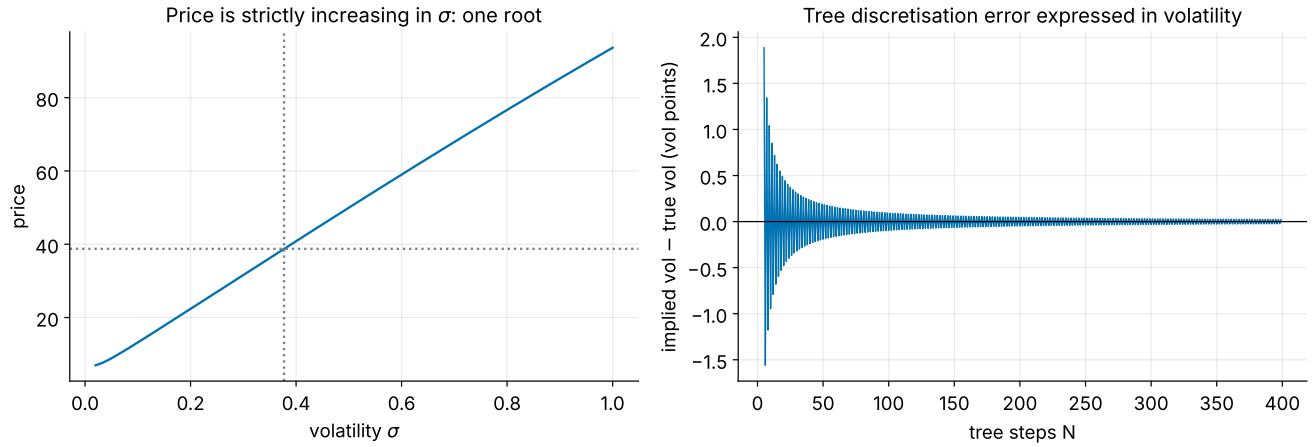


FIGURE 13. Left: the call price is strictly increasing in σ. Right: implied volatility of the CRR price minus the true volatility, for $N = 5, \dots, 399$.

profit is

$$\varepsilon = C_0 + \sum_{k=0}^{n-1} \Delta_k \big(e^{-rt_{k+1}} S_{t_{k+1}} - e^{-rt_k} S_{t_k} \big) - e^{-rT} (S_T - K)^+, \tag{6}$$

which is zero in continuous time. Table 11 and Figure 14 show that the standard deviation of ε decays like $n^{-1/2}$, with a fitted slope of -0.49. With daily rebalancing it is \$1.91, which is 4.9% of the premium, and with weekly rebalancing it is \$4.15, or 10.7%. The mean error is close to zero and the dispersion is the same, within simulation noise, when the stock is simulated with the real-world drift instead of the risk-neutral one. The hedge therefore works whatever the drift, which is the practical content of the statement that μ does not enter the price. The residual error is the sum of n gamma-weighted surprises of the squared return, each of size proportional to $1/n$, which explains the square-root law. Cutting the hedging risk in half therefore needs four times as many trades, and with transaction costs this is where hedging becomes a cost-benefit decision.

rebalances n	sd of hedging error (\$)	as % of premium	mean error (\$)	sd under real-world drift
4	14.187	36.6	-0.099	14.692
8	10.302	26.6	+0.073	10.521
16	7.391	19.1	+0.027	7.303
32	5.286	13.6	+0.040	5.270
52	4.152	10.7	-0.061	4.194
64	3.778	9.7	+0.022	3.750
126	2.653	6.8	+0.028	2.711
252	1.908	4.9	-0.004	1.925

TABLE 11. Standard deviation of the discounted hedging error of a short call against the number of rebalances n (risk-neutral paths; last column with real-world drift $\mu = 10\%$). Premium \$38.7493.

If the stock has realised volatility σ_{real} and the hedge uses σ_h, the discounted profit of the seller is, to leading order,

$$\varepsilon \approx \int_0^T e^{-rt}\,\tfrac{1}{2}\,\Gamma_t S_t^2\,(\sigma_h^2 - \sigma_{real}^2)\,dt, \tag{7}$$

so the seller loses when the realised variance exceeds the variance used to price and hedge. Table 12 compares the simulated mean profit with the integral in Equation (7) evaluated along the same paths. At a realised volatility of 55% the simulation gives \$−15.78 and the formula gives \$−15.72. At realised volatilities of 25% and 30% the sign reverses and the seller gains by comparable amounts, so the profit is close to linear in the variance gap. The size of the error is large compared with the discretisation error of Table 11: choosing the wrong volatility costs far more than trading weekly instead of daily. Section 5.13 repeats this experiment for the American put, where a wrong volatility also corrupts the exercise rule.

realised σ	simulated mean P&L	gamma formula	sd of P&L
25.0%	+11.704	+11.714	3.726
30.0%	+7.109	+7.116	2.803
37.7%	-0.013	+0.000	1.953
45.0%	-6.672	-6.685	3.583
55.0%	-15.781	-15.716	7.780

TABLE 12. Hedging with the wrong volatility ($\sigma_h = 37.73\%$, daily rebalancing, 10^4 paths): simulated mean profit of the seller against the gamma formula.

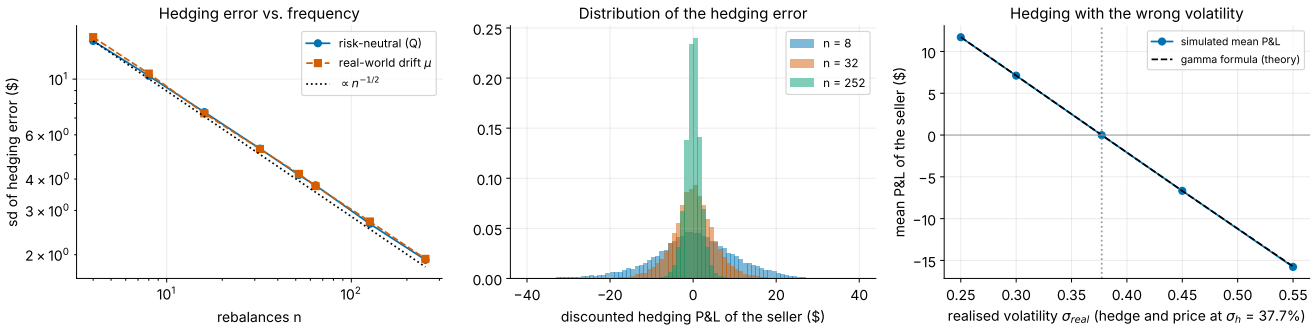


FIGURE 14. Left: standard deviation of the hedging error against n. Middle: distribution of the error for three values of n. Right: mean profit when the realised volatility differs from the hedging volatility.

For a single European call on one stock, a Leisen–Reimer tree or a smoothed and extrapolated CRR tree is orders of magnitude more accurate than Monte Carlo for the same run time. The work–precision comparison in Section 6 puts numbers on this. Monte Carlo wins when the payoff depends on the whole path or on many underlyings, because its error rate does not depend on the dimension, while the cost of a lattice grows exponentially with it. Variance reduction closes part of the gap: a control variate alone was worth a factor of 7.7 in variance here. The three routes to the Greeks agree, so the choice among them is a matter of cost and variance, not of correctness. The hedging error of a short call falls only like $n^{-1/2}$ with the number of rebalances,

and an error of 10 volatility points in the hedge volatility costs more than any reasonable change of frequency. Early exercise needs a decision at every date based on the value of waiting, which the tree computes naturally and Monte Carlo can approximate only with a regression, as Part C shows.

5. Part C: the American put

5.1. **Optimal stopping.** An American put may be exercised at any time $\tau \le T$ and then pays $(K - S_\tau)^+$. The holder chooses τ using only the information available, a stopping time, so under $\mathbb{Q}$ the fair value is the optimal stopping problem

$$V(t,x) = \sup_{\tau \in [t,T]} \mathbb{E}\Big[e^{-r(\tau - t)} (K - S_\tau)^+ \,\Big|\, S_t = x \Big]. \tag{8}$$

Two bounds follow at once: $V(t,x) \ge (K-x)^+$, by exercising immediately, and $V(t,x) \ge V^{Eur}(t,x)$, by choosing $\tau = T$. The American put is therefore never worth less than the European put or than its intrinsic value. The state space splits into a continuation region $C = \{V > (K-x)^+\}$ and a stopping region $D = \{V = (K-x)^+\}$, which for a put has the form $\{x \le b(t)\}$. The boundary $b(t)$ is a free boundary, because it is part of the unknown and has to be found together with V, and the optimal rule is $\tau^* = \inf\{t : S_t \le b(t)\}$. Equivalently, the discounted value $e^{-rt}V(t,S_t)$ is the smallest $\mathbb{Q}$-supermartingale that dominates the discounted payoff, the Snell envelope, and both the tree and the PDE are discretisations of this principle.

5.2. **The variational PDE and hedging.** Consider first a European claim and let $g(t,S_t) = e^{-rt}P(t,S_t)$ be a martingale. Assuming $g \in C^{1,2}$ and applying Itô's formula with $dS = rS\,dt + \sigma S\,dW$ gives

$$d\big(e^{-rt}P\big) = e^{-rt}\big(\partial_t P + rx\,\partial_x P + \tfrac{1}{2}\sigma^2 x^2 \partial_{xx} P - rP\big)\,dt + e^{-rt}\sigma x\,\partial_x P\,dW, \tag{9}$$

and a martingale has zero drift, so $(\mathcal{L} - r)P = 0$ with $P(T,x) = h(x)$, which is the Black–Scholes PDE. For the American put the discounted value $e^{-rt}V$ is only a supermartingale, so the drift in Equation (9) is non-positive everywhere. It must also dominate the obstacle $(K-x)^+$, and it is a martingale while the holder waits optimally, so the drift vanishes in the continuation region. Together these facts give the variational inequality, also called a linear complementarity problem:

$$(\mathcal{L} - r)V \le 0, \quad V \ge (K-x)^+, \quad \big(V - (K-x)^+\big)(\mathcal{L} - r)V = 0, \quad V(T,x) = (K-x)^+. \tag{10}$$

At every point either the obstacle binds or the PDE holds with equality, and in a single line the system reads $\min\{-(\mathcal{L}-r)V,\ V-(K-x)^+\} = 0$. In the continuation region $x > b(t)$ the PDE holds with equality, and in the stopping region $V = K - x$, where $(\mathcal{L}-r)(K-x) = -rx - r(K-x) = -rK$. This is negative for $r > 0$, so waiting loses money in the stopping region, which is exactly why the holder exercises there.

The boundary $b(t)$ is part of the unknown, and the condition that fixes it is smooth fit: $\partial_x V$ is continuous across the boundary, so $\partial_x V(t, b(t)) = -1$. Heuristically, if $\partial_x V$ jumped at the boundary, the stopping rule could be improved by moving the boundary slightly. A numerical solver never has to impose smooth fit, because the maximum in the tree recursion, or the projection in a finite-difference scheme, solves the obstacle problem directly. Because the market is complete, the second fundamental theorem of asset pricing implies that the claim exercised at τ^* can be replicated by holding $\Delta_t = \partial_x V(t, S_t)$ shares and the rest in the bond. This is the delta hedge of the European option with the American value function: the formula is the same, the function is different, and $\Delta = -1$ in the stopping region because there $V = K - x$. The replication is exact

up to τ^*, and if the holder exercises at any other time the seller is left with a surplus, so the hedge super-replicates every exercise date. Section 5.7 checks this statement numerically in the tree.

5.3. **The perpetual put.** If $T=\infty$ nothing depends on t, so the boundary is a constant $b(t)\equiv S^*$ and the system (10) becomes an ODE,

$$\tfrac12\sigma^2x^2V''+rxV'-rV=0\ (x>S^*),\qquad V=K-x\ (x\le S^*),\qquad V\ge(K-x)^+, \tag{11}$$

with V and V' continuous at S^*. Trying $V=Ax^{-\beta}$ with $\beta>0$, so that $V\to0$ as $x\to\infty$, turns the ODE into $(\beta+1)\big(\tfrac12\sigma^2\beta-r\big)=0$, hence $\beta=2r/\sigma^2$. In the notation $\kappa=r/\sigma^2$ and $\Delta=\sqrt{(\kappa-\tfrac12)^2+2\kappa}=\kappa+\tfrac12$ this reads $\beta=\kappa-\tfrac12+\Delta=2\kappa$. Matching the value and the slope at S^*, that is $A(S^*)^{-\beta}=K-S^*$ and $-\beta A(S^*)^{-\beta-1}=-1$, determines both constants:

$$S^*=\frac{\beta K}{\beta+1},\qquad V_\infty(x)=\frac{K}{\beta+1}\Big(\frac{S^*}{x}\Big)^{\beta}\quad(x>S^*). \tag{12}$$

A second derivation avoids smooth fit: exercising at the first time the price falls to a threshold b is worth $V_b(x)=(K-b)(b/x)^\beta$, because $\mathbb{E}[e^{-r\tau_b}]=(b/x)^\beta$. Maximising $(K-b)b^\beta$ over b gives $b=\beta K/(\beta+1)=S^*$, so smooth fit is nothing but the first-order condition of the stopping problem.

5.4. **The CRR tree with early exercise.** With $\Delta t=T/M_{bin}$, the CRR parameters of Section 4 and $V_{M,i}=(K-S_{M,i})^+$, the American put is priced by

$$V_{j,i}=\max\Big\{(K-S_{j,i})^+,\ e^{-r\Delta t}\big(p\,V_{j+1,i+1}+(1-p)\,V_{j+1,i}\big)\Big\}, \tag{13}$$

where the first term is the value of exercising now and the second the value of waiting. The exercise boundary $b(t_j)$ is the largest node at which exercising is strictly better than waiting, with a tolerance of 10^{-12} so that ties are resolved in favour of waiting. The tolerance matters for $r=0$, where exercise and continuation coincide deep in the money, and because the nodes of a CRR tree alternate between two grids on odd and even layers the boundary is plotted on even layers only. The hedge at node (j,i) is the ratio $\Delta_{j,i}=\frac{V_{j+1,i+1}-V_{j+1,i}}{S_{j+1,i+1}-S_{j+1,i}}$ of the changes in option value and stock price over one step. The optimal stopping time of a simulated path is the first date at which the path lies in the exercise region, and it is held to maturity if it never does. The class `CRRTree` extends `CRRBinomialModel` of Section 4: it adds the American recursion, a vectorised simulation of paths and the stopping rule, and inherits the lattice. The base class counts maturity in trading days, so the subclass passes `days=round(252*T)`, and each layer of the recursion is evaluated with array operations.

We show that, in the continuation region, one step of the recursion reproduces the PDE as $\Delta t\to0$. Write $y=\sigma\sqrt{\Delta t}$ so that $u=e^y$ and expand the value function at the two successor nodes. A Taylor expansion to order Δt gives the two successor values

$$V(t+\Delta t,xu)=V+V_t\Delta t+V_x\,x\big(\sigma\sqrt{\Delta t}+\tfrac12\sigma^2\Delta t\big)+\tfrac12V_{xx}x^2\sigma^2\Delta t+\dots \tag{14}$$

$$V(t+\Delta t,xd)=V+V_t\Delta t+V_x\,x\big(-\sigma\sqrt{\Delta t}+\tfrac12\sigma^2\Delta t\big)+\tfrac12V_{xx}x^2\sigma^2\Delta t+\dots \tag{15}$$

From $pu+(1-p)d=e^{r\Delta t}$ one obtains $2p-1=\frac{(r-\frac12\sigma^2)\sqrt{\Delta t}}{\sigma}+o(\sqrt{\Delta t})$. Hence the $\sqrt{\Delta t}$-terms contribute $(r-\tfrac12\sigma^2)x\,\Delta t\,V_x$, the $\tfrac12\sigma^2\Delta t\,xV_x$ terms complete this to $rxV_x\Delta t$, and $pV_u+(1-p)V_d=V+\big(V_t+rxV_x+\tfrac12\sigma^2x^2V_{xx}\big)\Delta t+o(\Delta t)$. Multiplying by $e^{-r\Delta t}=1-r\Delta t+\dots$ and requiring the right-hand side of Equation (13) to equal V gives $(\mathcal{L}-r)V=0$ in the continuation region, while in the stopping region the maximum sets $V=K-x$. Complementarity is therefore automatic,

the tree is an explicit finite-difference scheme for the variational inequality (10), and both give the same price as $M_{bin} \to \infty$.

5.5. **Results: price, boundary and hedge.** For the baseline $\kappa = 0.5$, so $\beta = 1$, $S^* = 5$ and $V_\infty(10) = 2.5$ (Figure 15). The perpetual put is worth 2.5 at the money, more than three times the one-year American put of Section 5.5. A numerical check confirms that V_∞ solves the system (11): the ODE residual is below 8×10^{-9} on $x > S^*$, the generator equals $-rK < 0$ on $x < S^*$, the value is above the payoff everywhere and the slope on the right of S^* is within $2 \cdot 10^{-4}$ of -1. The value and the payoff coincide at S^*. Since $\beta = 2r/\sigma^2$ and $S^*/K = \beta/(\beta+1)$, a higher interest rate raises β and moves S^*/K towards one, so one exercises earlier because the strike earns interest. A higher volatility lowers β and S^*/K, because the option to wait is worth more; Figure 16 shows both dependences. The level S^* is the price below which waiting no longer pays for its cost, and the perpetual put is the limit of the long-dated puts tested in Section 5.6.

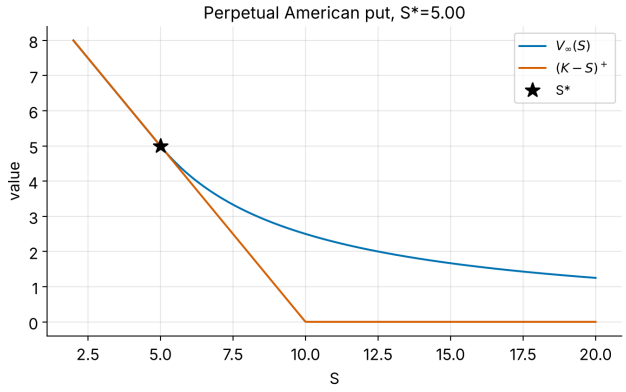


FIGURE 15. $V_\infty(S)$ and $S^* = 5$.

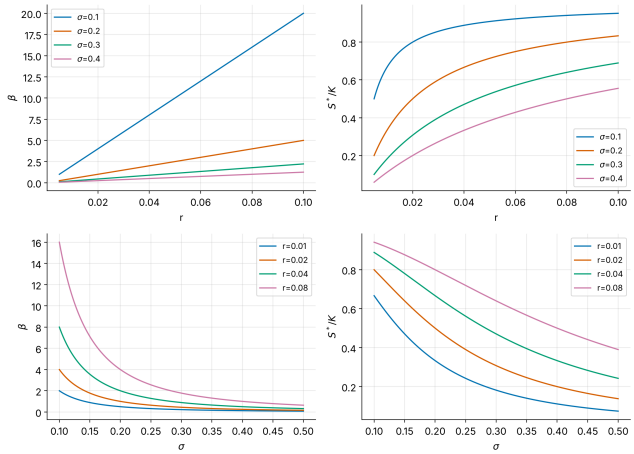


FIGURE 16. β and S^*/K against r and σ.

With $M = 2$ steps, $\Delta t = 0.5$, $u = e^{0.2\sqrt{0.5}} = 1.15191$, $d = 0.86812$, $p = 0.50012$ and $e^{-r\Delta t} = 0.99005$. The stock takes the values 10, then 8.6812 or 11.5191, then 7.5364, 10 or 13.2690. At the lower node of the first layer, where $S = 8.68$, exercising pays 1.3188 while waiting is worth $0.99005\,(0.50012 \cdot 0 + 0.49988 \cdot 2.4636) = 1.2193$. The holder therefore exercises early at this node, which is the first place where the American recursion differs from the European one. This lifts the price from the European value of 0.6034 to the American value of 0.6527. The hedge ratio at the root is $\Delta_{0,0} = \frac{0-1.3188}{11.5191-8.6812} = -0.4647$, so the replicating portfolio of the put is short about half a share, and the seller of the put hedges by holding that much.

Table 13 shows that the tree price increases from 0.7088 at $M_{bin} = 50$ to 0.7109 at $M_{bin} = 800$ and 0.7110 at $M_{bin} = 1600$. The successive changes halve each time M_{bin} doubles, so the ratio in the fourth column stays close to 2. A ratio of two is the signature of first-order convergence, $O(1/M_{bin})$, as for the European tree. Extrapolating, the limit is close to 0.7111, a value confirmed by independent methods in Section 5.8. The exercise boundary $b(t;1)$ in Figure 17 rises from about 7.4 near $t = 0$ to the strike at maturity, and the curves for the six step counts lie on top of each other. At $t = 0.5$ it equals 7.87 for the finest tree, and the column of Table 13 varies only by about one lattice spacing between step counts. With six months left the holder exercises once the stock has fallen about a fifth below the strike, and close to maturity the boundary approaches K.

This shape reflects the trade-off behind the problem: waiting forgoes interest on the strike, which is worth more than the remaining optionality when the option is deep in the money.

M_{bin}	$V^{bin}(0,S_0)$	change	ratio	$b(0.5;1)$
50	0.70877	–	–	7.97
100	0.70995	0.00117	–	7.87
200	0.71051	0.00057	2.07	7.75
400	0.71080	0.00029	1.97	7.87
800	0.71094	0.00014	2.04	7.86
1600	0.71101	0.00007	2.00	7.87

TABLE 13. CRR price convergence of the American put, $T = 1$: change, ratio of successive changes and boundary at $t = 0.5$.

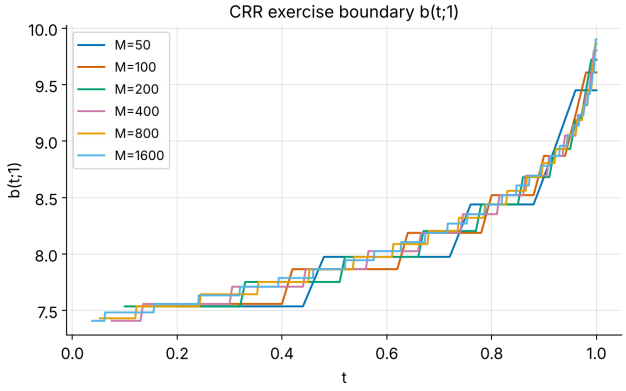


FIGURE 17. Exercise boundary $b(t;1)$.

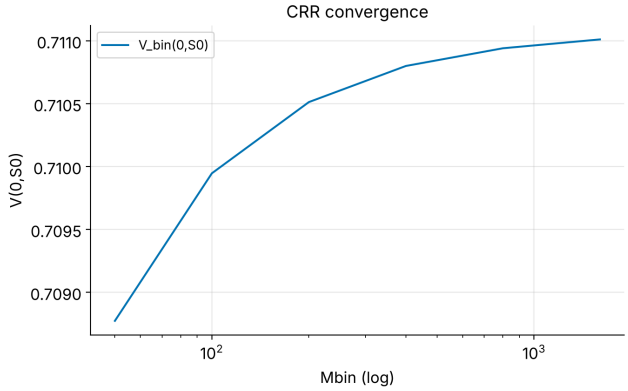


FIGURE 18. Convergence of $V^{bin}(0,S_0)$.

Path 1 in Figure 19 first enters the exercise region at $t = 0.48$ and is stopped there, whereas Path 2 never enters it and is held to maturity. Both paths were selected from a random search with fixed seed to illustrate the two cases. Along Path 1 the hedge ratio starts at -0.44 and moves to -0.99 as the path approaches the boundary, so the replicating portfolio ends up short almost one full share, and the seller of the put holds almost one full share (Figure 20). This is the smooth-fit condition seen from the hedger's side: the slope of the value function tends to -1 at the boundary. Along Path 2 the stock moves away from the strike, the put loses value, and the hedge ratio returns towards 0 near maturity. The hedge for an option that ends out of the money is therefore unwound without ever reaching the exercise region. A European put has no such plateau: its hedge ratio stays strictly above -1 at every spot and every date.

Higher volatility lowers the boundary: at $t = 0.5$ and $r = 2\%$ it equals 9.06, 7.86 and 6.68 for $\sigma = 10\%$, 20% and 30%. The option to wait is worth more when the stock is more volatile, so the holder is willing to wait longer, and the price rises with σ (Table 14). A higher interest rate raises the boundary, from 7.86 to 8.20 when r goes from 2% to 4% at $\sigma = 20\%$, because the strike earns interest once the option is exercised. The price falls with r for the same reason that a put loses value when interest rates rise. For $r = 0$ the boundary is empty in Table 14 because waiting never costs anything, so the American put equals the European put. Exercising early would give up optionality and gain no interest. The hedge ratio at the start, $\Delta(0,S_0)$, lies between -0.48 and

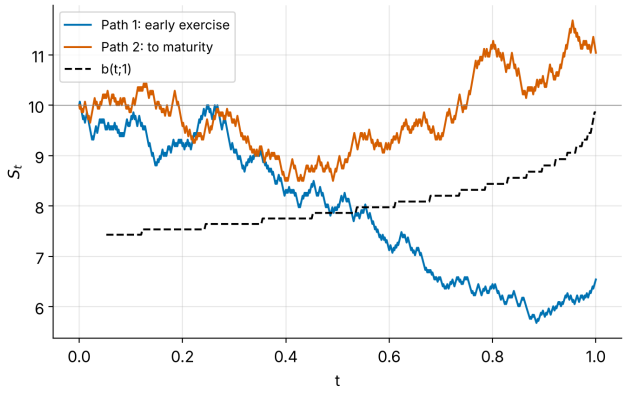


FIGURE 19. Two simulated paths and $b(t;1)$.



FIGURE 20. Hedging strategy $\Delta(t, S_t)$ along both paths.

-0.41 and depends only mildly on (σ, r). Higher volatility flattens $\Delta(t, \cdot)$ as a function of spot, and for $r > 0$ the plateau at -1 starts at the boundary, which is higher for larger r and smaller σ (Figure 22).

σ	r	$V(0, S_0)$	$b(0.5)$	$b(0.9)$	$\Delta(0, S_0)$
0.1	0	0.3987	–	–	-0.480
0.1	0.02	0.3224	9.06	9.45	-0.436
0.1	0.04	0.2665	9.25	9.52	-0.409
0.2	0	0.7963	–	–	-0.460
0.2	0.02	0.7109	7.86	8.80	-0.436
0.2	0.04	0.6403	8.20	8.93	-0.418
0.3	0	1.1920	–	–	-0.440
0.3	0.02	1.1011	6.68	8.09	-0.424
0.3	0.04	1.0227	7.27	8.26	-0.411

TABLE 14. Price, boundary and delta for $M_{bin} = 800$, $T = 1$. A dash means that the tree never exercises early.

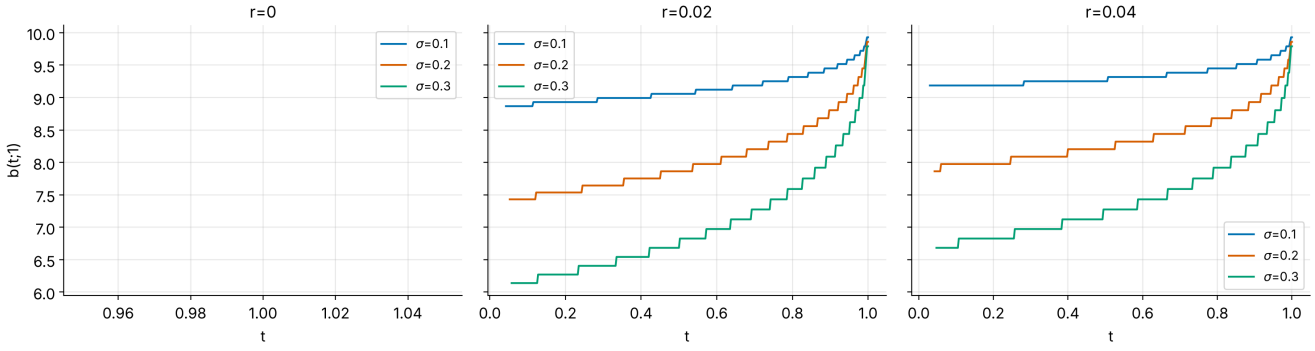


FIGURE 21. $b(t;1)$ for each (σ, r).

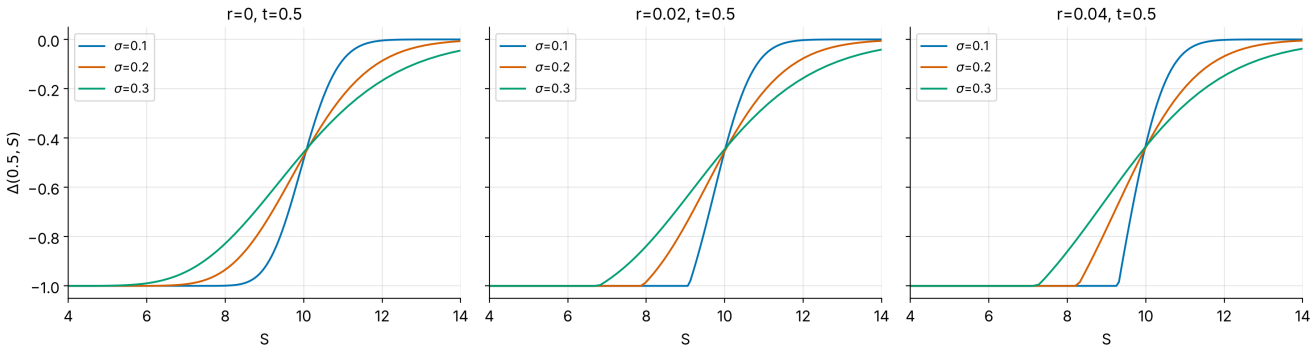


FIGURE 22. $\Delta(0.5, S)$ for each (σ, r).

5.6. **Long maturities and the perpetual limit.** With $M_{bin} = 800$ steps for every maturity, the price $V(0, S_0)$ increases with T towards the perpetual value of 2.5: it is 1.72 at $T = 10$, 2.06 at $T = 20$, 2.38 at $T = 50$ and 2.47 at $T = 100$. The curves $V(0, x)$ in Figure 24 approach $V_\infty(x)$ for all x. The boundary at date zero decreases from 5.8 at $T = 10$ to 5.1 at $T = 100$, towards $S^* = 5$. Plotted against the time to maturity $T - t$, the curves $b(t; T)$ for different T coincide and flatten at S^* (Figure 25). The row for $T = 1000$ breaks the pattern: it gives $b(0; 1000) = 5.5$, above the value 5.1 at $T = 100$, and a price that has stopped improving. An option that is almost perpetual cannot have a boundary that moves away from S^*, so the row must be a numerical artefact. The artefact is the lattice: with $M_{bin} = 800$ and $T = 1000$ the step is $\Delta t = 1.25$ and $u = 1.25$, far too coarse to resolve a boundary near 5, and the next paragraph repeats the experiment with a finer lattice.

T	$V^{bin}(0, S_0)$	$b(0; T)$	$\lvert V^{bin} - V_\infty \rvert$
10	1.7207	5.8	0.7793
20	2.0601	5.4	0.4399
50	2.3766	5.2	0.1234
100	2.4746	5.1	0.0254
1000	2.4847	5.5	0.0153
∞	2.5000	5.0	0

TABLE 15. Long maturities with $M_{bin} = 800$ for every T (first computation). The row $T = 1000$ is a lattice artefact.

Table 16 uses $M_{bin} = \max(800, 8T)$ steps, so that $\Delta t \leq 0.125$, and finds the boundary at date zero by bisection on the decision at the root, which determines $b(0; T)$ to the accuracy of the bisection instead of to a grid of step 0.1. The price and the boundary now increase and decrease monotonically with T up to $T = 200$. For $T = 500$ and $T = 1000$ the rows coincide, with $V = 2.4980$ and $b = 5.18$, so the gap of 2.0×10^{-3} to the perpetual value is no longer a maturity effect: it is a lattice effect. At $T = 1000$ the effect of maturity is of order e^{-40}. Table 17 refines the lattice at $T = 500$: the gap to 2.5 falls from 0.00303 to 0.00085 and the boundary comes down from 5.25 to 5.09 as Δt goes from 0.25 to 0.03. The price gap is not monotone in M_{bin} (it oscillates with the position of the nodes relative to S^*), whereas the boundary error decreases steadily. After subtracting the lattice floor of 0.0020, the distance $V_\infty(S_0) - V(0, S_0)$ decays roughly exponentially

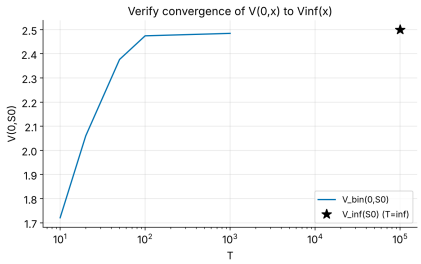


FIGURE 23. $V(0, S_0)$ against T.

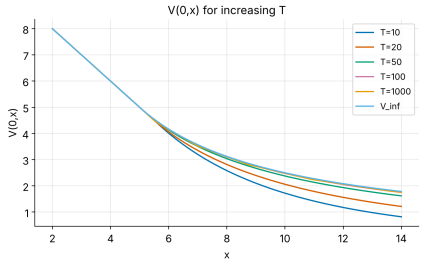


FIGURE 24. $V(0, x)$ against $V_\infty(x)$.

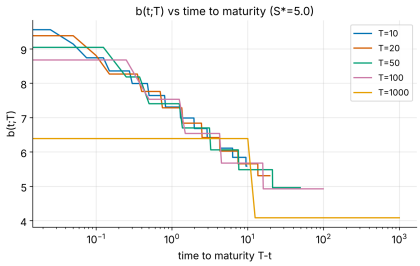


FIGURE 25. $b(t; T)$ against $T - t$.

in T, with a fitted rate of 0.040 per year between $T = 5$ and $T = 100$ (Figure 26). This rate is an empirical fit, not a derived constant, and it applies to this parameter set only.

T	M_{bin}	Δt	$V^{bin}(0, S_0)$	$b(0; T)$	$V_\infty - V$	$b - S^*$
2	800	0.003	0.9569	6.933	1.5431	1.933
5	800	0.006	1.3676	6.262	1.1324	1.262
10	800	0.013	1.7207	5.812	0.7793	0.812
20	800	0.025	2.0601	5.466	0.4399	0.466
50	800	0.062	2.3766	5.223	0.1234	0.223
100	800	0.125	2.4746	5.196	0.0254	0.196
200	1600	0.125	2.4966	5.180	0.0034	0.180
500	4000	0.125	2.4980	5.179	0.0020	0.179
1000	8000	0.125	2.4980	5.179	0.0020	0.179

TABLE 16. Long maturities with a refined lattice ($M_{bin} = \max(800, 8T)$) and the date-0 boundary found by bisection.

M_{bin}	Δt	$V^{bin}(0, S_0)$	$V_\infty - V$	$b(0; T)$	$b - S^*$
2000	0.2500	2.49697	0.00303	5.255	0.255
4000	0.1250	2.49805	0.00195	5.179	0.179
8000	0.0625	2.49915	0.00085	5.126	0.126
16000	0.0312	2.49915	0.00085	5.089	0.089

TABLE 17. Lattice refinement at $T = 500$: the remaining gap to the perpetual values $V_\infty = 2.5$ and $S^* = 5$ is a lattice effect.

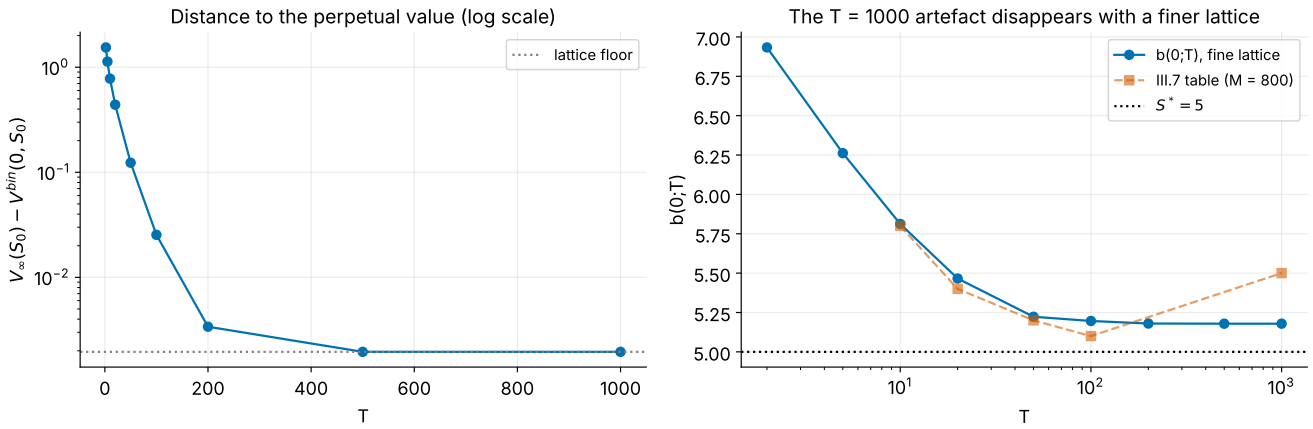


FIGURE 26. Left: distance to the perpetual value against T (log scale), with the lattice floor. Right: $b(0;T)$ with the refined and the original lattice.

5.7. **Replication in the tree.** We simulate 200 paths of the baseline tree with 800 steps and, at every date before exercise, compare the value of the replicating portfolio one step later with the option value at the node that was reached. The portfolio holds $\Delta_{j,i}$ shares and $B = V_{j,i} - \Delta_{j,i}S_{j,i}$ in the bond. The largest discrepancy over all hedge dates on all paths is 4×10^{-15}, which is rounding error. Replication is therefore exact in the tree up to the optimal exercise time. Inside the exercise region the identity fails by construction, because the holder is paid $K - S$, which exceeds the continuation value that the hedge replicates. The excess is the seller's surplus when the holder exercises, and at the boundary nodes we sampled it is tiny, of the order of 0.0002 in price units.

5.8. **Independent validators: finite differences, regression Monte Carlo, approximation.** We solve the variational inequality (10) on a uniform grid in $\ln x$ with Crank–Nicolson time stepping, started by four implicit half steps to smooth the kink of the payoff. The obstacle is enforced by the penalty method, which iterates on the set of nodes where the value is at the payoff, and the exercise boundary is read from that set. As a sanity check, the same code without the obstacle reproduces the Black–Scholes European put with an error of 9×10^{-6}. This tests the grid, the boundary conditions and the time stepping separately from the obstacle. Table 18 shows that the PDE price converges to 0.71107, against 0.71108 for the Richardson-extrapolated tree, a difference of 9.5×10^{-6}. The PDE converges faster than the tree and its finest grid takes about a second. Figure 27 overlays the exercise boundaries of the two methods, and they differ by at most 0.10 for $t < 0.97$. This is smaller than the spacing of the tree's nodes near the strike, so the two boundaries agree within the resolution of the tree.

Least-squares Monte Carlo works backward on simulated paths: at each exercise date it regresses the discounted future cash flow on a polynomial of the spot, using only in-the-money paths, and exercises when the payoff exceeds the fitted continuation value. The estimate is biased low, because the regression policy is not the optimal one, and it approaches the Bermudan value as the basis grows. With 50 exercise dates and $2 \cdot 10^5$ paths the estimate is 0.7095 with standard error 0.0018, against 0.7105 for a Bermudan tree on the same dates. The difference is within one standard error, as it is for the other date counts in Table 19. The last column of Table 19 shows that the gap between the American and the Bermudan value halves as the number of exercise dates doubles. Restricting exercise to fifty dates therefore costs about 0.0006 in value, which is below

grid points	time steps	$V(0, S_0)$	error vs tree (Richardson)	CPU (s)
201	50	0.710224	-8.6e-04	0.02
401	100	0.710826	-2.5e-04	0.06
801	200	0.711001	-8.0e-05	0.14
1601	400	0.711054	-2.7e-05	0.43
3201	800	0.711071	-9.5e-06	1.49

TABLE 18. Crank–Nicolson with penalty for the American put: grid refinement against the Richardson-extrapolated tree.

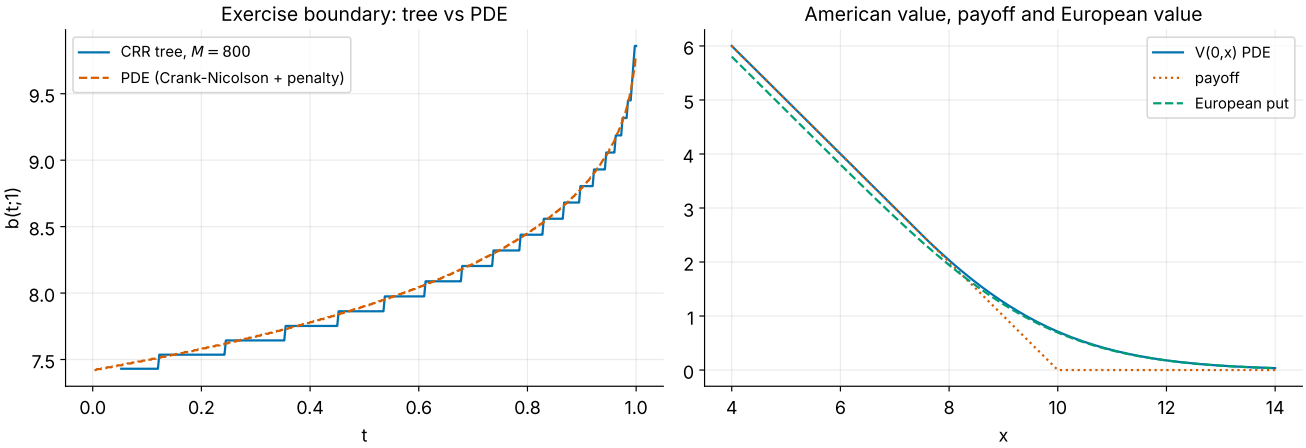


FIGURE 27. Left: exercise boundary from the tree ($M = 800$) and from the PDE solver. Right: value, payoff and European value from the PDE solver.

the standard error of the regression estimate. The Barone–Adesi–Whaley approximation, which approximates the early-exercise premium by a power of the spot, gives 0.7100 at the baseline, an error of -0.0011, and a critical price of 7.55 against the tree boundary of about 7.4 at the start. Table 20 shows that its absolute error grows with the maturity and reaches about one cent for the three-year put, which is why it serves for quick estimates and not for final prices.

exercise dates	LSM (s.e.)	Bermudan tree	LSM − Bermudan	American − Bermudan
10	0.7110 (0.0019)	0.7085	+0.0024	+0.0026
25	0.7092 (0.0019)	0.7100	-0.0008	+0.0011
50	0.7095 (0.0018)	0.7105	-0.0010	+0.0006
100	0.7106 (0.0018)	0.7108	-0.0002	+0.0003

TABLE 19. Longstaff–Schwartz ($2 \cdot 10^5$ paths) against a Bermudan tree with the same exercise dates.

S_0	T	European (BS)	American (tree)	BAW	BAW − tree
8	0.25	1.9547	2.0000	2.0000	+0.0000
10	0.25	0.3733	0.3768	0.3761	-0.0007
12	0.25	0.0129	0.0130	0.0131	+0.0002
8	1	1.9447	2.0323	2.0248	-0.0075
10	1	0.6936	0.7111	0.7100	-0.0011
12	1	0.1762	0.1791	0.1807	+0.0016
8	3	2.0351	2.2123	2.2019	-0.0103
10	3	1.0637	1.1277	1.1298	+0.0021
12	3	0.5222	0.5458	0.5532	+0.0074

TABLE 20. Barone-Adesi–Whaley approximation against the Richardson-extrapolated tree ($K = 10$, $\sigma = 20\%$, $r = 2\%$).

5.9. **The early-exercise premium.** The early-exercise premium is the difference between the American and the European value computed on the same lattice, so that the lattice error cancels and the premium is exactly zero when there is no early exercise. It is zero for $r = 0$ and for a call without dividends. At the money in the baseline the premium is 0.0175, which is 2.5% of the European put. Figure 28 shows that it grows as the put goes deeper into the money and with higher interest rates and lower volatility, which are the conditions under which the holder wants to collect the strike early. The map in Figure 29 shows the premium in price units and as a percentage of the European put. Over the region plotted, spot 6 to 14 and volatility 10% to 50%, the relative premium reaches about 13% of the European value. The premium at the money also grows with the maturity, as the right panel of Figure 28 shows. For American options put–call parity becomes the inequality $S_0 - K \le C - P_A \le S_0 - Ke^{-rT}$, and the numbers are $0.0000 \le 0.1805 \le 0.1980$, so the American put respects the bounds.

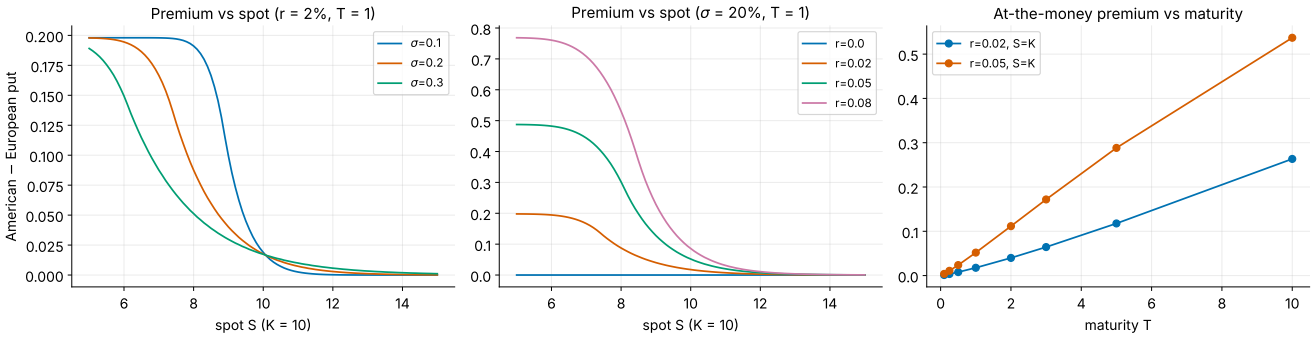


FIGURE 28. American minus European put on the same lattice, against spot (left, middle) and maturity (right).

5.10. **Dividends.** A continuous dividend yield q lowers the risk-neutral drift of the stock to $r - q$, so the tree uses $p = (e^{(r-q)\Delta t} - d)/(u - d)$ while discounting stays at r. The subclass `CRRTreeGen` changes only this probability and the payoff, so the boundary, the hedge and the stopping rule are inherited from `CRRTree`. With $q = 0$ the tree never exercises the American call early, and its price equals the European price on the same lattice. This is the classical result that early exercise of a call is never optimal without dividends. Table 21 shows that the premium grows with the

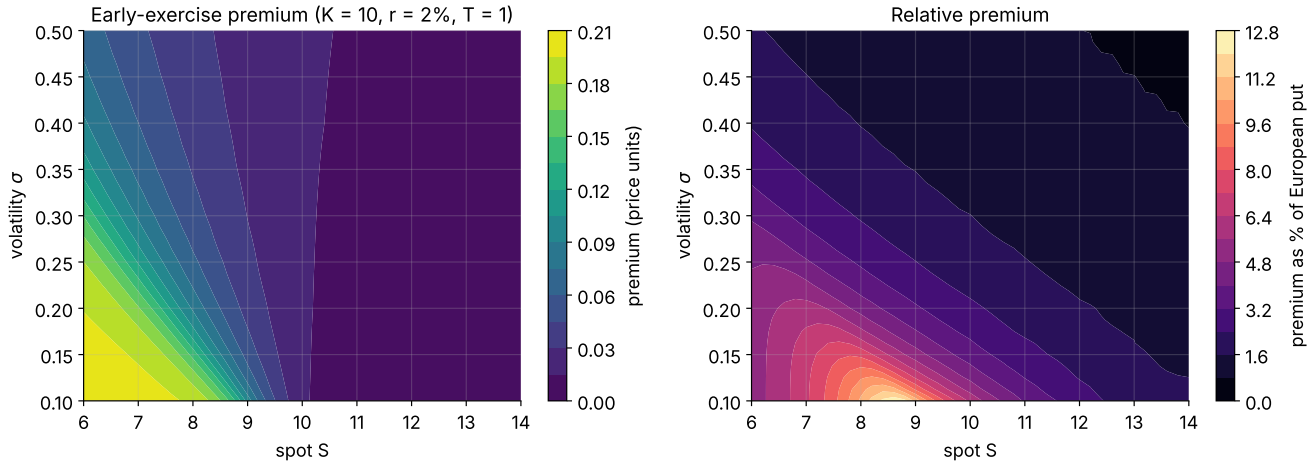


FIGURE 29. Early-exercise premium in price units (left) and as a percentage of the European put (right).

yield, from zero to 0.229 at $q = 6\%$. The exercise region of a call is $\{x \geq b(t)\}$ with $b(t)$ above the strike, and Figure 30 shows that the boundary comes down as the yield rises. The intuition is that waiting costs the dividends qS per year and earns interest rK on the deferred strike payment, so early exercise becomes attractive when the yield is large relative to the interest rate.

dividend yield q	European call (BS)	American call (tree)	premium	$b(0.5)$
0%	14.2313	14.2276	+0.0000	–
2%	13.0203	13.0167	+0.0000	288.8
4%	11.8833	11.9258	+0.0461	162.9
6%	10.8186	11.0438	+0.2287	146.5

TABLE 21. European and American call with continuous dividend yield q ($S_0 = K = 100$, $\sigma = 30\%$, $r = 5\%$, $T = 1$, $M = 800$). The premium is American minus European on the same lattice.

5.11. **An American put on the NVIDIA inputs.** Put–call parity applied to the Black–Scholes call of Part B gives a European put of \$31.9207, which agrees with the direct formula to 3×10^{-14}. The American put is worth \$32.5674, an early-exercise premium of \$0.6467 or 2.03% of the European put. Read as a European put, the American price implies a volatility of 38.43%, which is 0.70 volatility points above the 37.73% of the model. A trader who quotes the American put in implied volatility would therefore quote it slightly higher than the European one for the same stock. The delta of the American put is -0.407 against -0.395 for the European put, and the vega is almost the same. Rho and theta differ more, because the right to exercise early shortens the effective life of the option and reduces its exposure to the interest rate. At $t = 0.5$ the exercise boundary is \$147.8, which is 61.9% of the strike, far below the 79% of the baseline because the volatility is almost twice as high. Figure 31 shows the boundary divided by the strike for five volatility and rate regimes, and Figure 32 the value, delta and gamma of the American and European put against the spot.

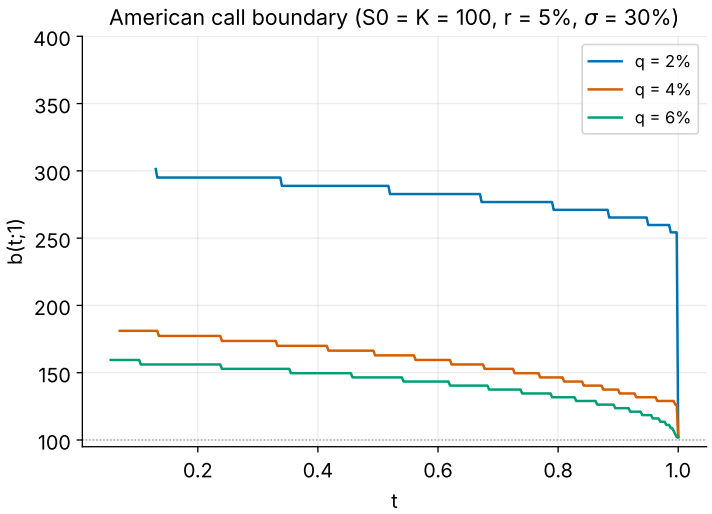


FIGURE 30. Exercise boundary of the American call for three dividend yields.

	European put (BS)	American put (tree)
price	31.9207	32.5674
Δ	-0.39531	-0.40708
Γ	0.0042732	0.0044968
vega	92.007	92.069
Θ (per year)	-13.69	-14.5
ρ	-126.36	-99.365

TABLE 22. European and American put on the NVIDIA inputs of Part B (Greeks of the American put from the tree, $N = 1000$).

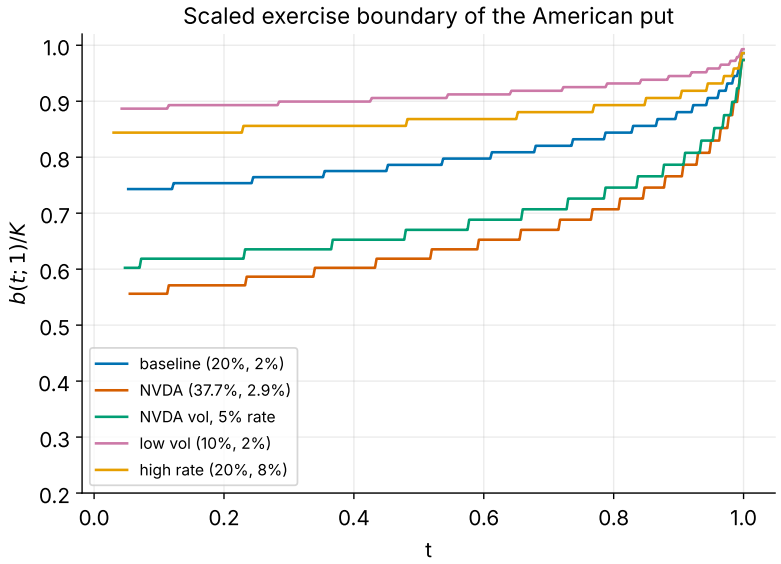


FIGURE 31. Exercise boundary divided by the strike for five volatility and rate regimes.

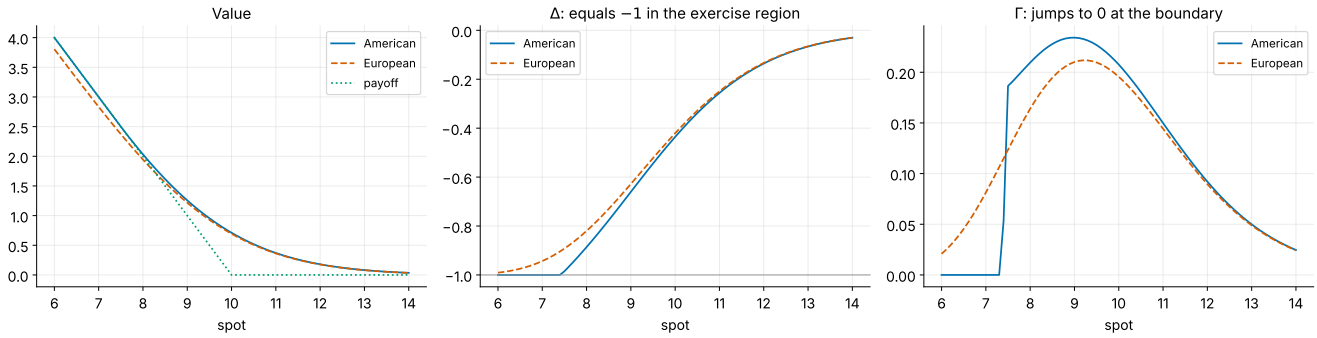


FIGURE 32. Value, delta and gamma of the American and European put against spot at $t = 0$ (baseline).

5.12. **Profit and loss of the buyer.** The buyer pays the fair value $V(0, S_0) = 0.7109$ and exercises at the first date on which the path enters the exercise region. Paths are simulated in the tree with the pricing weights $(p, 1-p)$, because under the pricing measure the real-world drift μ plays no role. The mean profit is -0.004 with a standard error of 0.008, consistent with zero, as it must be by the definition of the fair value. A vectorised run with 10^5 paths gives -0.0041 with a standard error of 0.0026, which confirms it. The standard deviation of the profit is 0.83, the mean stopping time is 0.877 years and the probability of exercising before maturity is 0.52. Roughly half of the paths are therefore stopped early. The distribution in Figure 33 has a point mass at $-V_0$ for the paths that end out of the money and a long right tail, and Table 23 repeats the experiment on the nine (σ, r) combinations. The spread grows with σ and shrinks with r, the probability of early exercise rises with σ, and for $r = 0$ the option is never exercised early, so that $\tau = T$.

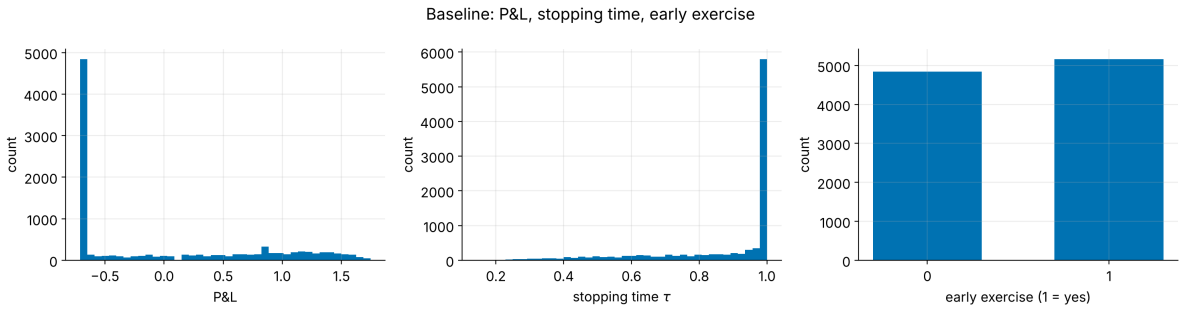


FIGURE 33. Baseline profit and loss, stopping time and early-exercise indicator, 10,000 paths.

σ	r	mean P&L (sd)	mean τ	p_{ex}
0.1	0	-0.005 (0.546)	1.000	0.000
0.1	0.02	-0.002 (0.391)	0.854	0.473
0.1	0.04	-0.003 (0.324)	0.808	0.444
0.2	0	-0.009 (1.031)	1.000	0.000
0.2	0.02	-0.004 (0.831)	0.877	0.516
0.2	0.04	-0.004 (0.742)	0.843	0.496
0.3	0	-0.012 (1.461)	1.000	0.000
0.3	0.02	-0.001 (1.236)	0.883	0.543
0.3	0.04	-0.007 (1.135)	0.855	0.529

TABLE 23. Profit and loss and exercise statistics, 10,000 paths each.

Under a real-world drift of $\mu = 5\%$ the mean profit of the buyer is -0.094, and at $\mu = 10\%$ it is -0.246. A drift above the risk-free rate makes the stock rise faster than under the pricing measure, so the put pays out less often than the price assumes. The probability that the buyer loses the entire premium rises from 48.8% under the pricing measure to 54.4% at $\mu = 5\%$. The probability of exercising early falls to 0.46, and the mean stopping time rises to 0.894 years. The buyer's loss is capped at the premium, so the tail risk is on the seller's side, where the loss equals the buyer's profit. The 99% quantile of the buyer's profit is 1.63 under the real-world measure and 1.63 under the pricing measure, and the expected shortfall at 95% is 1.54 against 1.56, so the

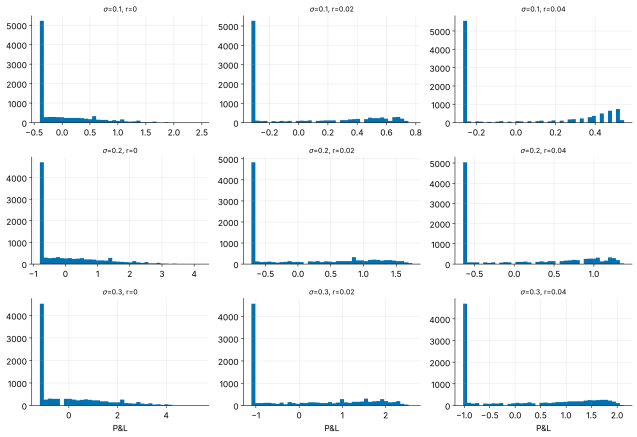


FIGURE 34. P&L histograms (rows σ, columns r).

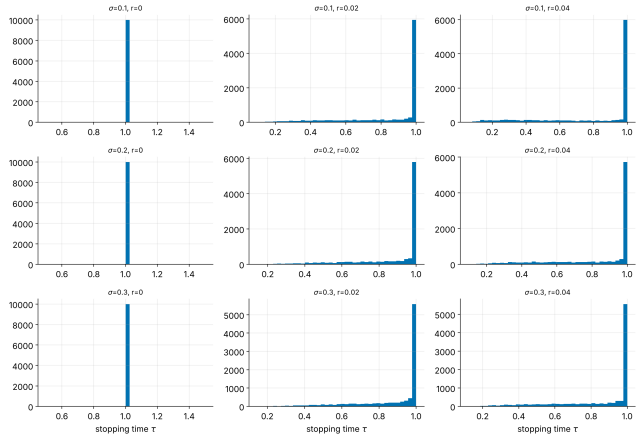


FIGURE 35. Stopping times (rows σ, columns r).

right tail hardly moves. The real-world drift changes how often the put pays out, not how much it pays, which is why the mean moves while the tail does not. A seller who hedges dynamically is insulated from the drift, as Section 4.9 showed for the call, whereas an unhedged seller keeps the drift risk.

Measure	mean	sd	P(lose premium)	95% quantile	99% quantile	$\mathrm{ES}_{95\%}$	mean τ	p_{ex}
risk-neutral, $\mu = r = 2\%$	-0.003	0.834	48.8%	1.463	1.632	1.561	0.876	0.512
real world, $\mu = 5\%$	-0.094	0.807	54.4%	1.407	1.632	1.542	0.894	0.456
real world, $\mu = 10\%$	-0.246	0.739	64.1%	1.349	1.577	1.498	0.922	0.359
real world, $\mu = -5\%$	+0.241	0.869	35.3%	1.520	1.633	1.601	0.825	0.647

TABLE 24. Buyer's profit and loss under the pricing measure and under four real-world drifts (10^5 paths each); the buyer pays the $\mathbb{Q}$-price and exercises on the $\mathbb{Q}$-boundary. The tail columns are those of the buyer's profit, which is the seller's loss.

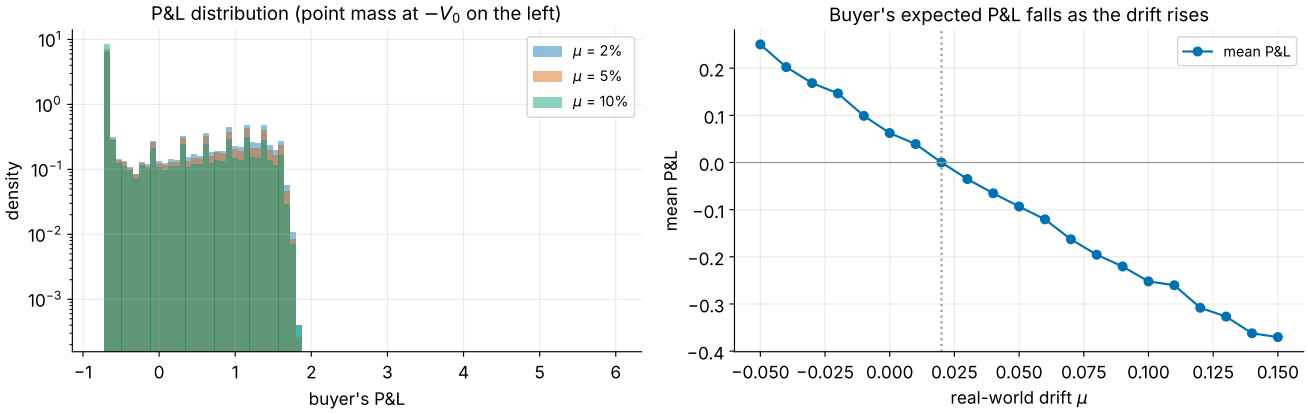


FIGURE 36. Left: distribution of the buyer's profit under three drifts (log density). Right: mean profit against the real-world drift.

5.13. **Mis-specified volatility.** Because the boundary $b(t;T)$ decreases in σ, the 20% boundary lies below the optimal one when the realised volatility is lower than 20%, so the buyer exercises too late. When the realised volatility is higher the 20% boundary lies above the optimal one, and the buyer exercises too early. At a realised volatility of 10% the mean stopping time is 0.977 against 0.854 for the optimal rule, and the buyer loses 0.016 relative to optimal exercise. Waiting too long costs a modest amount because the option is still close to the money. At 30% the mean stopping time is 0.766 against 0.883, and the loss relative to optimal exercise is 0.036. Exercising too early therefore costs more than exercising too late by a factor of more than two in this example. The mean profit itself, -0.41 at 10% and $+0.35$ at 30%, is driven mostly by the price mismatch, since the option is worth less or more than the buyer paid. The differences relative to the optimal rule are estimated with a standard error of up to 0.007 at 10,000 paths, so the loss at 10% is only about two standard errors, which the next section tightens with ten times more paths.

σ_{real}	mean P&L	sd	mean τ	p_{ex}	ΔP&L vs optimal (s.e.)	$\Delta\tau$ vs optimal
0.10	-0.407	0.471	0.977	0.411	-0.016 (0.003)	+0.123
0.15	-0.201	0.686	0.936	0.454	-0.004 (0.003)	+0.067
0.20	-0.004	0.831	0.877	0.516	+0.000 (0.000)	+0.000
0.25	+0.184	0.942	0.825	0.549	-0.006 (0.004)	-0.056
0.30	+0.353	1.002	0.766	0.590	-0.036 (0.007)	-0.118

TABLE 25. Following the $\sigma = 20\%$ boundary when the realised volatility differs; Δ is relative to the optimal rule of the realised volatility on the same paths.

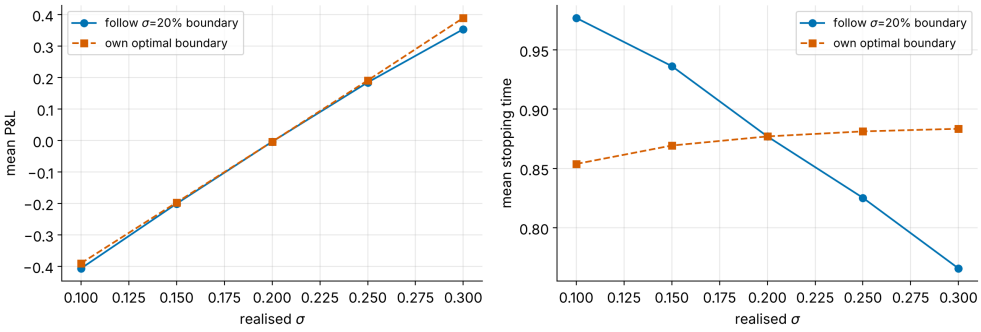


FIGURE 37. Mean profit and mean stopping time against the realised volatility.

The buyer's mean profit splits into the premium mismatch, the difference between the value under the realised volatility and the price paid, and the policy regret, the loss from using the wrong boundary. The first term is first-order in the volatility gap, roughly the vega of the put (about 3.90 per unit of volatility) times the gap, and the second is second-order. The regret matrix in Figure 38 is strongly asymmetric: the largest regret, 0.326, occurs when the price is set at 10% and the realised volatility is 30%, so that the buyer exercises too early. In the opposite corner, priced at 30% and realised at 10%, the buyer exercises too late and the regret is only 0.017. The asymmetry has a simple reason: exercising too early gives up an option whose value is large when volatility is high, whereas waiting too long costs only the interest on the strike and some time value. The ratio between the two corners is about twenty. Each cell uses 100 000 paths, so the

standard error of the regret is at most 0.0034, which is small compared with the large entries and comparable with the small ones. In the corner priced at 10% and realised at 30% the buyer's mean profit is +0.45, mostly because the option was bought cheaply, which shows that the premium mismatch dominates the regret.



FIGURE 38. Rows: priced volatility; columns: realised volatility. Left: buyer's mean profit. Middle: policy regret. Right: mean stopping time of the followed rule minus the optimal one (positive: too late).

priced σ_p	real. 10%	real. 15%	real. 20%	real. 25%	real. 30%
10%	+0.000 / 0.000	+0.170 / 0.022	+0.293 / 0.095	+0.393 / 0.189	+0.452 / 0.326
15%	-0.203 / 0.010	-0.002 / 0.000	+0.183 / 0.012	+0.337 / 0.051	+0.466 / 0.118
20%	-0.403 / 0.015	-0.203 / 0.006	-0.000 / 0.000	+0.188 / 0.006	+0.358 / 0.031
25%	-0.600 / 0.016	-0.403 / 0.011	-0.199 / 0.003	-0.002 / 0.000	+0.191 / 0.003
30%	-0.796 / 0.017	-0.601 / 0.014	-0.397 / 0.007	-0.199 / 0.002	-0.001 / 0.000

TABLE 26. Mean profit / policy regret for each pair of priced (rows) and realised (columns) volatility, 10^5 paths per cell.

The CRR tree, a finite-difference solver for the variational inequality and least-squares Monte Carlo agree with each other within their numerical errors, and the tree approaches the closed-form perpetual put as the maturity and the number of steps grow. The one row that disagreed, $T = 1000$ with 800 steps, was a lattice artefact, and the refined computation says so. Early exercise adds a few percent to an at-the-money put and much more deep in the money, and it is zero for $r = 0$ and for calls without dividends. For the NVIDIA put it adds about 2.03% to the European price. The buyer's expected profit under the real-world measure depends on the drift, which the price does not. A wrong volatility corrupts the exercise rule asymmetrically, because exercising too early costs much more than exercising too late. In practice the tree is the natural tool for prices, boundaries and hedges, a PDE solver is better when accuracy is needed in one or two dimensions, regression Monte Carlo takes over when the dimension is high, and Barone–Adesi–Whaley serves for a quick estimate.

6. Comparison of methods

Table 27 shows that the vectorised CRR tree with $N = 252$ reproduces the price of the list-based class, with an error of -0.035, in 0.6 ms instead of 12 ms. The Leisen–Reimer tree with $N = 101$

has an error of \$0.0002 in 0.2 ms, and CRR with smoothing and Richardson extrapolation at $N = 252$ has an error of \$0.0000. Plain Monte Carlo with 10^5 samples has a standard error of \$0.215 (the realised error is \$0.209) and takes 3.5 ms. The control variate brings the standard error to \$0.082 at the same cost, and scrambled Sobol points with 2^{16} samples reach an error of \$0.0013 in 2.6 ms. The Euler–Maruyama simulation with 252 steps takes about 398.1 ms for the same 10^5 samples, roughly two orders of magnitude more than the exact sampler, and has the same standard error. For a European payoff on geometric Brownian motion the extra cost buys nothing, because the exact terminal law is known; the scheme earns its place when the dynamics are not solvable. The order of the rows is stable across machines, but the absolute times are not and should be read as orders of magnitude.

Method	price	error vs BS	s.e.	CPU (ms)
Black-Scholes closed form	38.7493	+0.0000	–	0.2
CRR tree, $N = 252$ (class, lists)	38.7142	-0.0350	–	11.8
CRR tree, $N = 252$ (vectorised)	38.7142	-0.0350	–	0.6
CRR tree, $N = 1000$ (vectorised)	38.7404	-0.0088	–	3.4
CRR + smoothing + Richardson, $N = 252$	38.7493	+0.0000	–	2.5
Leisen-Reimer tree, $N = 101$	38.7491	-0.0002	–	0.2
Monte Carlo plain, $M = 10^5$ (exact sampler)	38.5401	-0.2092	0.2155	3.5
Monte Carlo control variate, $M = 10^5$	38.6810	-0.0682	0.0821	3.4
Monte Carlo Euler $N = 252$, $M = 10^5$	38.6841	-0.0652	0.2156	398.1
Scrambled Sobol QMC, 2^{16} points	38.7479	-0.0013	–	2.6

TABLE 27. European call: every method on one footing (single core; the CPU time is the fastest of several runs for the quick methods).

Figure 39 plots the error of each method against the CPU time it uses, on logarithmic axes. The tree errors fall with slope about -1 (CRR) or -2 (Leisen–Reimer) in the number of steps, the Monte Carlo standard errors fall with slope $-\frac{1}{2}$ in the time, and the Sobol points have a much steeper slope because the integral is one-dimensional. To bring the error below \$0.001, plain CRR needs $N = 10\,000$ steps and 94 ms, CRR with smoothing and Richardson needs $N = 201$ and 2 ms, and the Leisen–Reimer tree needs $N = 41$ and 0.10 ms. The gap between the first and the last is therefore a factor of several hundred in time. Plain Monte Carlo needs about 4.6×10^9 samples for a standard error of \$0.001, which is about 162 seconds by extrapolation, and the control variate needs about 21 seconds. Scrambled Sobol points reach the same accuracy with 131 072 points in a few milliseconds. This ranking is specific to a one-dimensional European payoff, and it reverses for many underlyings or for path-dependent payoffs, where lattices become infeasible and Monte Carlo keeps its rate.

Table 28 shows that the tree with $M = 1600$ steps is within 0.00007 of the extrapolated reference and takes 51 ms. The list-based class needs 24 ms for 800 steps against 15 ms for the vectorised pricer, and the reference itself costs 533 ms. The finite-difference solver on a 801×200 grid has an error of 0.00008 in 132 ms, and on a 3201×800 grid an error of 0.00001 in 1361 ms. In one dimension the PDE solver is therefore at least as accurate as the tree for comparable time, and it gives the boundary and the Greeks from the same grid. Least-squares Monte Carlo has an error of 0.0016 against a standard error of 0.0019 and takes 1402 ms, so it is the slowest method here, and its error is of the size of its own statistical noise. The Barone–Adesi–Whaley approximation

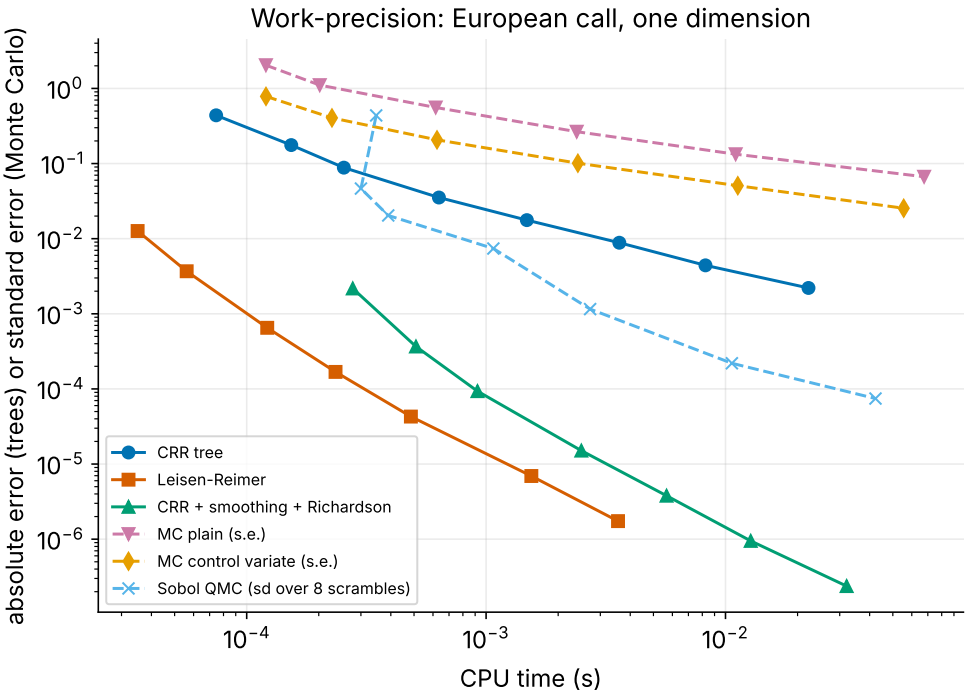


FIGURE 39. Work–precision diagram for the European call: absolute error of the trees and standard error of the Monte Carlo estimators against CPU time.

is off by 0.0011 but takes 1 ms, which is why it remains useful for screening. Least-squares Monte Carlo is worth having nonetheless, because its cost does not grow exponentially with the number of underlyings, which is the case where the tree and the grid fail.

Method	price	error vs reference	s.e.	CPU (ms)
European put (Black-Scholes), for reference	0.69359	-0.01749	–	0
CRR tree, $M = 800$ (class, lists)	0.71094	-0.00014	–	24
CRR tree, $M = 800$ (vectorised)	0.71094	-0.00014	–	15
CRR tree, $M = 1600$ (vectorised)	0.71101	-0.00007	–	51
CRR tree + Richardson ($M = 3200, 6400$): reference	0.71108	+0.00000	–	533
PDE, Crank-Nicolson + penalty, 801×200	0.71100	-0.00008	–	132
PDE, Crank-Nicolson + penalty, 3201×800	0.71107	-0.00001	–	1361
Longstaff-Schwartz, 50 dates, $2 \cdot 10^5$ paths	0.71267	+0.00159	0.0019	1402
Barone-Adesi-Whaley approximation	0.70999	-0.00109	–	1

TABLE 28. American put (baseline): every method against the Richardson-extrapolated tree.

For a vanilla European option the closed form is the answer when it exists, and otherwise a Leisen–Reimer tree or a smoothed and extrapolated CRR tree is faster and more accurate than any simulation. Monte Carlo serves there only as an independent cross-check. For an American option on one underlying a CRR tree with Richardson extrapolation delivers the price, the exercise boundary and the hedge in tens of milliseconds, and a PDE solver is the better choice when high accuracy is needed. The Barone–Adesi–Whaley formula is good enough for screening and for

initialising a root-finder. For path-dependent payoffs or several underlyings, Monte Carlo with variance reduction is the default, with a regression step such as Longstaff–Schwartz if early exercise is allowed. The cost of a tree or a grid grows exponentially with the dimension and rules them out beyond two or three underlyings. Sensitivities come from the same lattice or grid as the price where possible, and from pathwise or likelihood-ratio estimators in Monte Carlo. A stress of the volatility input is more informative than any refinement of the numerics, because in both parts of this report the effect of a wrong volatility dwarfs the numerical error of a reasonable method.

7. Conclusions

The discrete and the continuous language agree in the limit: the scaled random walk converges to Brownian motion at rate $N^{-1/2}$ in the Kolmogorov distance, and the CRR price converges to the Black–Scholes price at rate $1/N$ with a sign that depends on the parity of N. The sign pattern is a property of the kink at the strike and can be removed. For a vanilla payoff a well-chosen lattice beats Monte Carlo by orders of magnitude in accuracy per unit of time, and variance reduction recovers only a part of the gap. The strength of Monte Carlo is its generality, not its accuracy. The Greeks can be obtained from the closed form, from the tree and from Monte Carlo, and the three agree. The error of a discrete delta hedge falls only like $n^{-1/2}$ in the number of rebalances, and a wrong hedging volatility costs far more than a coarser schedule. The point to remember is that, for a reasonable method, the volatility that goes into the model matters more than the numerical scheme that processes it.

The CRR tree is an explicit finite-difference scheme for the variational inequality, and the tree, a finite-difference solver and least-squares Monte Carlo agree on the price of the American put. The perpetual put is the limit of the long-dated put, and the one case that seemed to contradict this, a thousand-year maturity on a coarse lattice, was an artefact of the lattice. Early exercise adds a few percent at the money and more deep in the money, and it needs a positive interest rate for a put and a dividend for a call. The boundary falls when volatility rises and rises when the interest rate rises, which is what the cost and the value of waiting predict. The price does not depend on the real-world drift but the buyer's expected profit does, and a seller who does not hedge carries that dependence. A wrong volatility corrupts the exercise rule, and the loss is asymmetric: exercising too early costs much more than exercising too late. In practice the volatility input and the exercise rule should be monitored together, because an error in the first propagates into the second.

The model assumes a constant volatility, whereas real markets show an implied-volatility smile and term structure, so a local- or stochastic-volatility model would be the natural next step. The tree would then no longer recombine so simply, and the PDE would gain a second dimension. The inputs are simplified: a historical volatility instead of an implied one, a Canadian rate for a US-dollar stock, and no dividends for NVIDIA. In practice each of these would be replaced by market data, and the sensitivities of Section 4.7 say how much the price would move. There are no transaction costs and no jumps in the model, and discrete hedging is studied only for the call. Jump or gap risk would leave a residual that no rebalancing frequency removes. Natural next steps are to calibrate to an option chain, to add a discrete dividend schedule, and to extend the regression Monte Carlo to a basket of underlyings.

Appendix A. Derivations

A.1. **Itô's formula and the solution of GBM.** The rules of Itô calculus follow from the quadratic variation of Brownian motion: $(dt)^2 = 0$, $dt\,dW = 0$ and $(dW)^2 = dt$. For a twice differentiable f and $dS = \mu S\,dt + \sigma S\,dW$ it gives, using $(dS)^2 = \sigma^2 S^2\,dt$,

$$df(S_t) = f'(S_t)\,dS_t + \tfrac{1}{2} f''(S_t)\,\sigma^2 S_t^2\,dt. \tag{16}$$

With $f = \ln$ one has $f' = 1/S$ and $f'' = -1/S^2$, so $d\ln S_t = (\mu - \tfrac{1}{2}\sigma^2)\,dt + \sigma\,dW_t$. Integrating from 0 to t and exponentiating gives $S_t = S_0 \exp\big((\mu - \tfrac{1}{2}\sigma^2)t + \sigma W_t\big)$. To check this, apply Itô's formula to $g(t, W) = S_0 e^{(\mu - \frac{1}{2}\sigma^2)t + \sigma W}$: $dg = (\mu - \tfrac{1}{2}\sigma^2) g\,dt + \sigma g\,dW + \tfrac{1}{2}\sigma^2 g\,dt = \mu g\,dt + \sigma g\,dW$. Hence $\ln S_t$ is normal with mean $\ln S_0 + (\mu - \tfrac{1}{2}\sigma^2)t$ and variance $\sigma^2 t$, and since $\mathbb{E}[e^{\sigma W_t}] = e^{\sigma^2 t/2}$ one gets $\mathbb{E}[S_t] = S_0 e^{\mu t}$ and $\operatorname{Var}[S_t] = S_0^2 e^{2\mu t}\big(e^{\sigma^2 t} - 1\big)$. The variance is what sets the size of the Monte Carlo standard errors of Section 4.

A.2. **The Black–Scholes formula by direct integration.** Under $\mathbb{Q}$ write $S_T = S_0 e^{(r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T} Z}$ with $Z \sim N(0,1)$, so that $C = e^{-rT}\mathbb{E}[(S_T - K)^+]$. The payoff is positive exactly when $Z > -d_2$, where $d_2 = \frac{\ln(S_0/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$. Splitting the expectation gives $C = e^{-rT}\big(\mathbb{E}[S_T \mathbf{1}_{\{Z > -d_2\}}] - K\,\mathbb{P}(Z > -d_2)\big)$. By symmetry $\mathbb{P}(Z > -d_2) = \Phi(d_2)$, so the second term is $K e^{-rT}\Phi(d_2)$. For the first term, completing the square gives $e^{\sigma\sqrt{T} z}\varphi(z) = e^{\sigma^2 T/2}\varphi(z - \sigma\sqrt{T})$, so

$$\mathbb{E}[S_T \mathbf{1}_{\{Z > -d_2\}}] = S_0 e^{rT} \int_{-d_2}^{\infty} \varphi(z - \sigma\sqrt{T})\,dz = S_0 e^{rT}\,\Phi(d_2 + \sigma\sqrt{T}) = S_0 e^{rT}\,\Phi(d_1). \tag{17}$$

Combining the two terms gives Equation (5): $\Phi(d_2)$ is the risk-neutral probability that the call ends in the money, and $\Phi(d_1)$ is the same probability under the measure that takes the stock as numeraire.

A.3. **The Black–Scholes PDE and the hedging argument.** Let $V(t, S_t)$ be the value of the option and consider a portfolio that is short one option and long Δ shares, so that $\Pi = \Delta S - V$. We choose Δ so that the portfolio has no exposure to the Brownian motion. Itô's formula gives $dV = V_t\,dt + V_S\,dS + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\,dt$ under the real-world dynamics (1). With $\Delta = V_S$ the terms in dS cancel in $d\Pi = \Delta\,dS - dV$, which leaves the deterministic increment $d\Pi = -\big(V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\big)\,dt$. A portfolio without risk must earn the riskless rate, so $d\Pi = r\Pi\,dt = r(SV_S - V)\,dt$. Equating the two expressions gives the Black–Scholes PDE

$$V_t + rSV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} - rV = 0. \tag{18}$$

The real-world drift μ has disappeared from the equation, which is the continuous-time form of the statement that the price does not depend on μ. With the terminal condition $V(T, S) = (S - K)^+$ the solution is, by the Feynman–Kac formula, the discounted risk-neutral expectation $e^{-r(T-t)}\mathbb{E}^{\mathbb{Q}}[(S_T - K)^+ \mid S_t = S]$ of Section 2.

A.4. **Convergence of the CRR tree to geometric Brownian motion.** In the CRR tree the terminal log-price is $\ln S_N = \ln S_0 + \sigma\sqrt{\Delta t}\,(2B - N)$ with $B \sim \operatorname{Bin}(N, p)$. Its mean is therefore $N\sigma\sqrt{\Delta t}\,(2p - 1)$ and its variance is $4N\sigma^2\Delta t\,p(1-p)$, where $N\Delta t = T$. With $y = \sigma\sqrt{\Delta t}$, expanding $e^{r\Delta t} = 1 + r\Delta t + \dots$ and $e^{\pm y} = 1 \pm y + \tfrac{1}{2}y^2 + \dots$ in $p = \frac{e^{r\Delta t} - e^{-y}}{e^{y} - e^{-y}}$ gives

$$p = \frac{1}{2} + \frac{(r - \frac{1}{2}\sigma^2)\sqrt{\Delta t}}{2\sigma} + O(\Delta t). \tag{19}$$

Hence the mean tends to $N\sigma\sqrt{\Delta t}\cdot\frac{(r-\frac12\sigma^2)\sqrt{\Delta t}}{\sigma}=(r-\frac12\sigma^2)T$. Since $p\to\frac12$ the variance tends to σ^2T. The increments are bounded by $\sigma\sqrt{\Delta t}$, so the Lindeberg condition holds and the central limit theorem for triangular arrays gives a normal limit with these parameters, which is the law of $\ln S_T$ under geometric Brownian motion. The payoff is continuous and has linear growth and the tree has uniformly bounded moments, so the tree price converges to the Black–Scholes price, while the rate $1/N$ and the parity effect require a finer analysis of the distribution of the lattice around the strike.

A.5. **Pathwise and likelihood-ratio Greeks.** We want $\partial_\theta C(\theta)$ for $C(\theta)=e^{-rT}\mathbb{E}[f(S_T(\theta))]$, where θ is the spot, the volatility or the rate. There are two ways to move the derivative inside: differentiate the sample path, or differentiate the density of S_T. The pathwise method requires f to be Lipschitz and S_T to be differentiable in θ, and then $\partial_\theta\mathbb{E}f(S_T)=\mathbb{E}[f'(S_T)\,\partial_\theta S_T]$. For the call $f'(s)=\mathbf{1}_{\{s>K\}}$, which exists except at a single point that has probability zero. Since $\partial S_T/\partial S_0=S_T/S_0$ and $\partial S_T/\partial\sigma=S_T(\sqrt{T}Z-\sigma T)$, the pathwise estimators are

$$\hat\Delta=e^{-rT}\mathbf{1}_{\{S_T>K\}}\frac{S_T}{S_0},\qquad \hat{\mathcal{V}}=e^{-rT}\mathbf{1}_{\{S_T>K\}}\,S_T\big(\sqrt{T}Z-\sigma T\big). \tag{20}$$

The likelihood-ratio method writes $\mathbb{E}f(S_T)=\int f(s)\,g(s;\theta)\,ds$ and differentiates the density, $\partial_\theta\mathbb{E}f(S_T)=\mathbb{E}[f(S_T)\,\partial_\theta\ln g(S_T;\theta)]$, which needs no smoothness of f. For the lognormal density the score with respect to the spot is $Z/(S_0\sigma\sqrt{T})$, so $\Delta=e^{-rT}\mathbb{E}\big[(S_T-K)^+Z/(S_0\sigma\sqrt{T})\big]$. For the second derivative we apply the likelihood-ratio idea to the pathwise delta, which gives $\Gamma=e^{-rT}\mathbb{E}\big[\mathbf{1}_{\{S_T>K\}}\frac{S_T}{S_0^2}\big(\frac{Z}{\sigma\sqrt{T}}-1\big)\big]$, and in general the likelihood-ratio estimators have larger variance for smooth payoffs but remain valid for discontinuous ones.

A.6. **The hedging-error identity.** Let $V(t,S)$ solve the Black–Scholes PDE (18) with hedging volatility σ_h, and let the stock follow $dS=\mu S\,dt+\sigma_r S\,dW$ with realised volatility σ_r. The seller receives $V(0,S_0)$ and holds $\Delta_t=V_S(t,S_t)$ shares. Itô's formula for the discounted value under the true dynamics gives

$$d\big(e^{-rt}V\big)=e^{-rt}\Big[\big(V_t+\mu SV_S+\tfrac12\sigma_r^2S^2V_{SS}-rV\big)dt+\sigma_rSV_S\,dW\Big]. \tag{21}$$

Substituting $V_t=rV-rSV_S-\frac12\sigma_h^2S^2V_{SS}$ from the PDE turns the coefficient of dt into $(\mu-r)SV_S+\frac12(\sigma_r^2-\sigma_h^2)S^2V_{SS}$. The first term is exactly the drift of the hedge: the gain from holding $\Delta=V_S$ shares is $\Delta\,d(e^{-rt}S)=e^{-rt}V_S\big((\mu-r)S\,dt+\sigma_rS\,dW\big)$. Subtracting the gains from the change in value leaves $d(e^{-rt}V)-\Delta\,d(e^{-rt}S)=e^{-rt}\frac12(\sigma_r^2-\sigma_h^2)S^2\Gamma\,dt$, with $\Gamma=V_{SS}$. Integrating from 0 to T and using $V(T,S_T)=(S_T-K)^+$, the seller's discounted profit is $\varepsilon=\int_0^T e^{-rt}\frac12(\sigma_h^2-\sigma_r^2)S_t^2\Gamma_t\,dt$, which is Equation (7).

A.7. **Least-squares Monte Carlo.** With exercise dates $t_1<\cdots<t_n=T$ the value satisfies $V_n(x)=(K-x)^+$ and $V_j(x)=\max\{(K-x)^+,C_j(x)\}$ with the continuation value $C_j(x)=e^{-r\Delta t}\,\mathbb{E}[V_{j+1}(S_{j+1})\mid S_j=x]$. This is the tree recursion with the one-step expectation replaced by a conditional expectation under the simulated law. The conditional expectation is approximated by a regression of the discounted realised cash flow on a polynomial basis of the spot, $C_j(x)\approx\sum_{k=0}^d\beta_kx^k$, fitted on the paths that are in the money at t_j. Only in-the-money paths are used because exercise is never optimal otherwise, and this concentrates the fit where the decision is made. Working backward from the last date, a path exercises at t_j if its payoff exceeds the fitted continuation value, and its cash flow is then replaced by the payoff. The estimate is the average of the discounted cash flows, compared with the immediate payoff at the start. Because the fitted

rule is a feasible exercise policy, its expected payoff is a lower bound for the Bermudan value, which is why the estimator is biased low. Using the same paths for the fit and for the valuation adds a small upward bias of order $1/M$, so in practice the estimate is low-biased up to noise, and the accuracy is controlled by the basis and by the number of exercise dates.

A.8. **The Barone-Adesi–Whaley approximation.** Write the American put as $P_A = P_E + \varepsilon$, the European put plus the early-exercise premium. Both P_A and P_E satisfy the Black–Scholes PDE in the continuation region, hence so does ε. Barone-Adesi and Whaley set $\varepsilon = \kappa(T) f(S)$ with $\kappa = 1 - e^{-rT}$ and neglect a term that is small, which turns the PDE into an Euler equation $S^2 f'' + NSf' - \frac{M}{\kappa} f = 0$ with $M = 2r/\sigma^2$ and $N = 2(r-q)/\sigma^2$. Its decaying solution is a power of the spot, $f = A_1 S^{q_1}$, where $q_1 = \frac{1}{2}\big[-(N-1) - \sqrt{(N-1)^2 + 4M/\kappa}\big] < 0$. Value matching and smooth fit at the critical price S^{**} then fix both S^{**} and A_1. The result is

$$P_A(S) = P_E(S) + A_1 \Big(\frac{S}{S^{**}}\Big)^{q_1} \ (S > S^{**}), \qquad A_1 = -\frac{S^{**}}{q_1}\Big[1 - e^{(b-r)T}\Phi\big(-d_1(S^{**})\big)\Big], \tag{22}$$

where S^{**} solves $K - S^{**} = P_E(S^{**}) - \big[1 - e^{(b-r)T}\Phi(-d_1(S^{**}))\big] S^{**}/q_1$ and $b = r - q$. The neglected term vanishes only if κ were constant in time, so the approximation degrades as the maturity grows, as Table 20 shows. The formula also has a single critical price, whereas the true boundary depends on time, which is why it is best at short and medium maturities.

A.9. **The hitting-time identity for the perpetual put.** Let S follow geometric Brownian motion with drift r under $\mathbb{Q}$, start at $x > b$ and let τ_b be the first time it hits b. We claim that $\mathbb{E}[e^{-r\tau_b}] = (b/x)^\beta$ with $\beta = 2r/\sigma^2$. The process $M_t = e^{-rt} S_t^{-\beta}$ is a martingale, because $f(x) = x^{-\beta}$ solves $\frac{1}{2}\sigma^2 x^2 f'' + rxf' - rf = 0$, which is exactly the statement that the dt-coefficient in Itô's formula for $e^{-rt} f(S_t)$ vanishes. The coefficient is $f \cdot \big[\frac{1}{2}\sigma^2\beta(\beta+1) - r\beta - r\big] = f \cdot (\beta+1)\big(\frac{1}{2}\sigma^2\beta - r\big) = 0$. Before τ_b the price is at least b, so $0 < M_{t\wedge\tau_b} \le b^{-\beta}$ is bounded and the optional stopping theorem applies. It gives

$$x^{-\beta} = \mathbb{E}\big[M_{\tau_b}\big] = b^{-\beta}\,\mathbb{E}\big[e^{-r\tau_b}\big] \implies \mathbb{E}\big[e^{-r\tau_b}\big] = (b/x)^\beta. \tag{23}$$

On the event $\{\tau_b = \infty\}$ the discount factor $e^{-r\tau_b}$ is zero, so the identity holds without further conditions. It follows that exercising at the first time the price falls to b is worth $(K-b)(b/x)^\beta$, and maximising over b gives the boundary $S^* = \beta K/(\beta+1)$ of Section 5.3.

Appendix B. Additional figures: the CRR trees

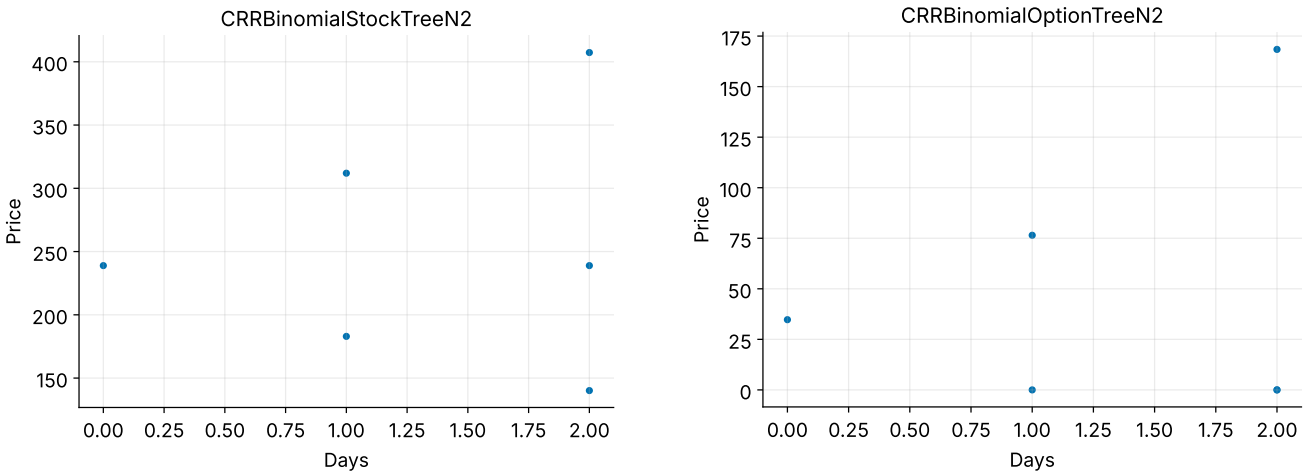


Figure 40. CRR stock tree (left) and option tree (right), $N = 2$.

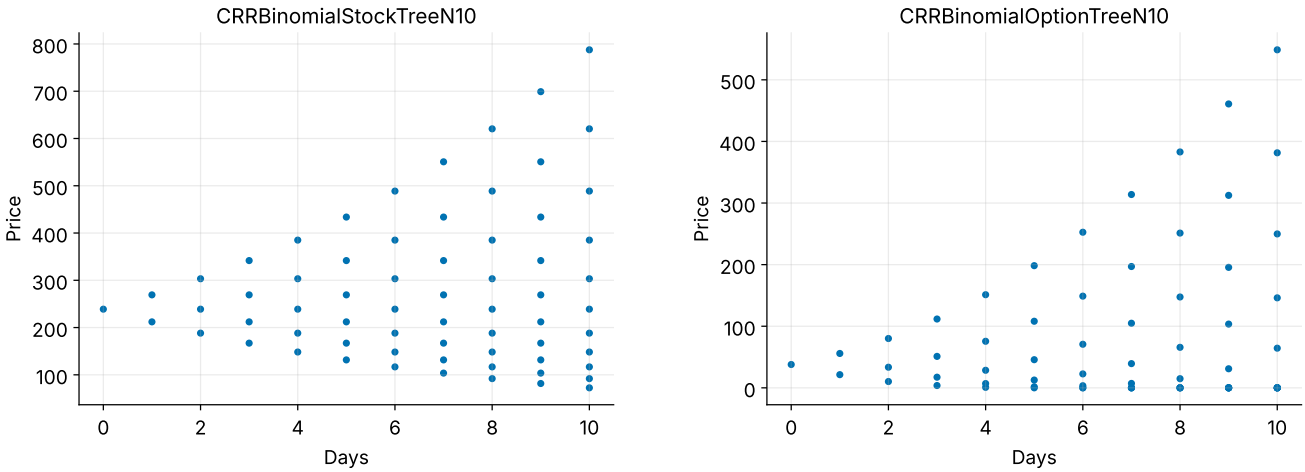


Figure 41. CRR stock tree (left) and option tree (right), $N = 10$.

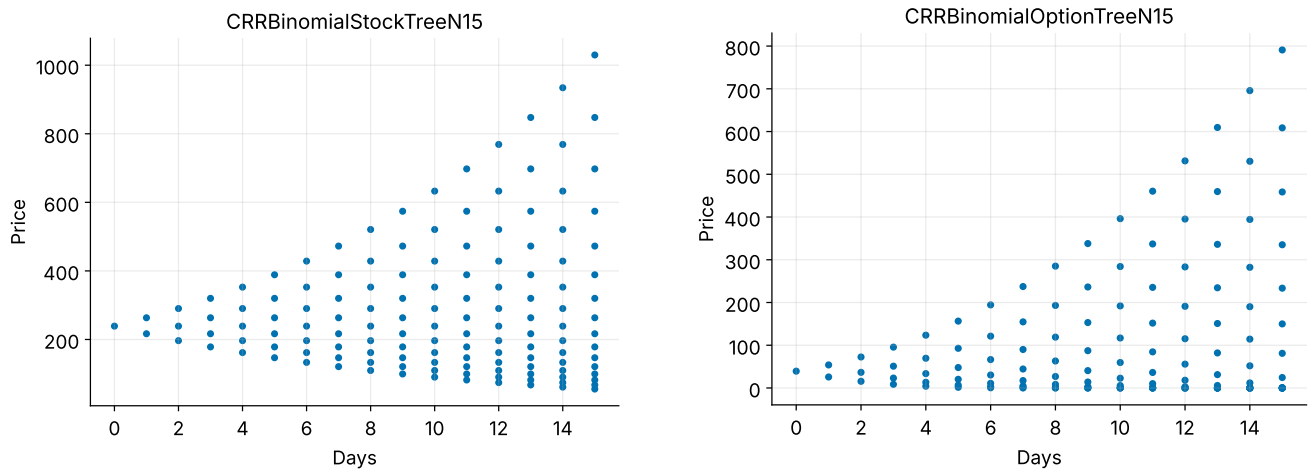


FIGURE 42. CRR stock tree (left) and option tree (right), $N = 15$.

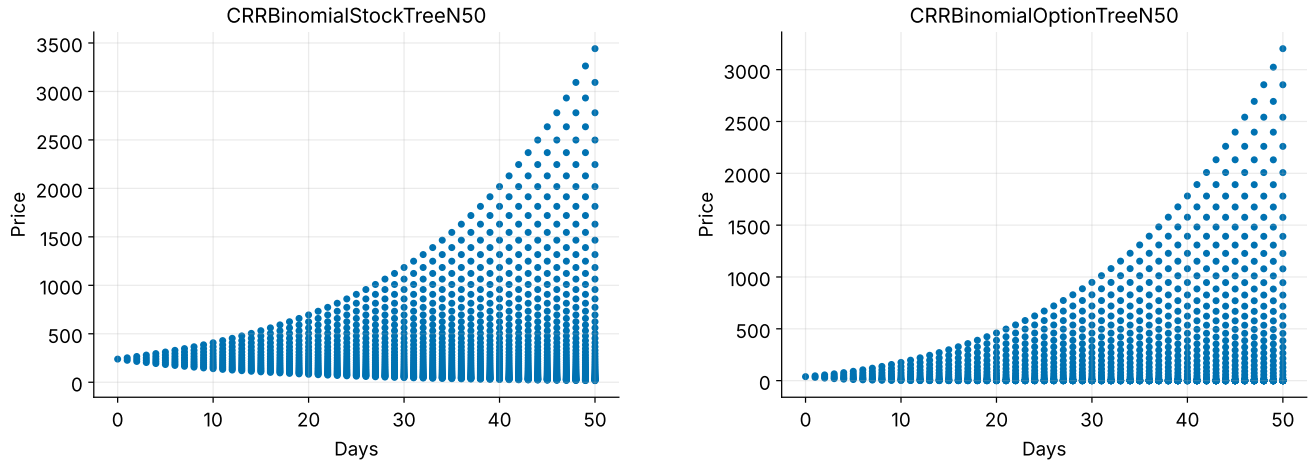


FIGURE 43. CRR stock tree (left) and option tree (right), $N = 50$.

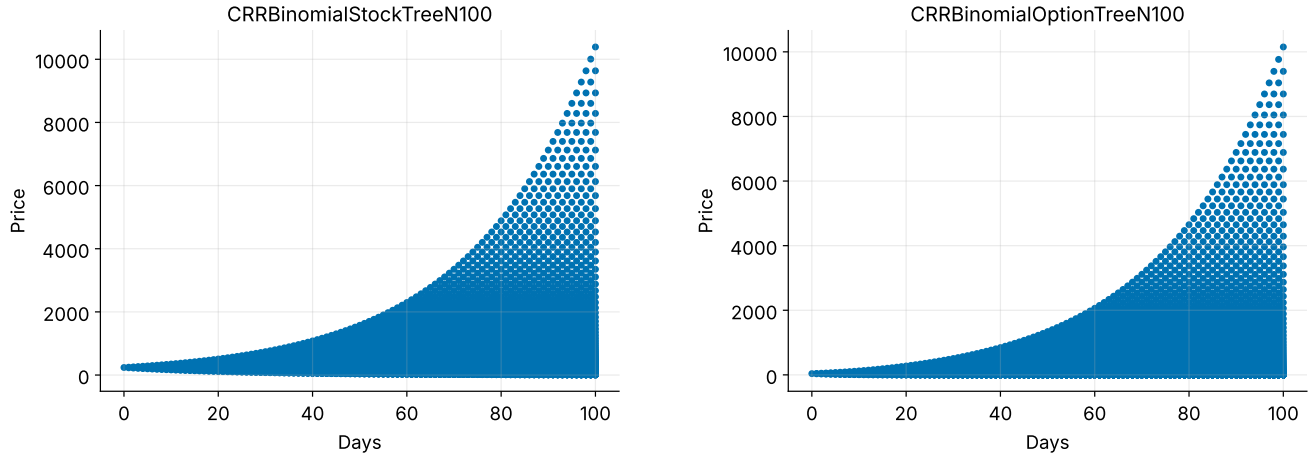


FIGURE 44. CRR stock tree (left) and option tree (right), $N = 100$.

Appendix C. Code

The listing shows the hand-written classes and functions exactly as used in the notebook, followed by the extensions that are built on top of them. The full notebook (with the experiments) is `notebooks/option_pricing.ipynb`.

```
# Hand-written classes and functions, exactly as used in the notebook
import math, random
import numpy as np
import matplotlib.pyplot as plt

class BrownianMotion():
    def __init__(self, N1 = 10_000, time_horizon = [0,1]):
        self.position = 0
        self.path = [0, ]
        self.N1 = N1
        self.time_horizon = time_horizon

    # Run BrownianMotion for one step
    # (independent from previous moves)
    def run(self, step, dim):
        self.position += random.gauss(mu = 0, sigma = step) # only support 1D for now
        self.path.append(self.position)

    def simulate(self):
        steps = self.N1
        interval = self.time_horizon

        dt = (interval[1] - interval[0]) / steps
        for _ in range(steps):
            self.run(step = math.sqrt(dt), dim = 1)
        return self.path

class symmetric_random_walk(BrownianMotion):
    def __init__(self, N2 = 10, time_horizon = [0,1]):
        super().__init__(N1 = N2, time_horizon = time_horizon)
        self.N2 = N2

    def run(self, step, dim):
        self.position += random.choice([step, -step]) # proba 1/2
        self.path.append(self.position)

    def simulate(self):
        steps = self.N2
        interval = self.time_horizon
        dt = (interval[1] - interval[0]) / steps
        for _ in range(steps):
            self.run(step = math.sqrt(dt), dim = 2)
        return self.path

class CRRBinomialModel():
    def __init__(self, initial_stock_value = 1, rate = risk_free_CAD_rate, volatility =
       annualized_252_volatility, steps = 100, days = 30, strike_price = 3):
        self.strike_price = strike_price
        self.initial_stock_value = initial_stock_value
        self.days = days
        self.rate = rate
        self.volatility = volatility
        self.steps = steps
        self.T = days / 252
        self.stepsize = self.T / steps
        self.u = math.exp(volatility * math.sqrt(self.stepsize)) # u = e^{sigma sqrt(dt)}
        self.d = 1 / self.u
        self.risk_neutral_p = (math.exp(rate * self.stepsize) - self.d) / (self.u - self.d)
         # p = (e^{r dt} - d)/(u - d)
        self.risk_neutral_q = 1 - self.risk_neutral_p
        self.stock_tree = []
        self.option_tree = []

    def clear_tree(self):
        self.stock_tree = []
        self.option_tree = []

    def generate_stock_tree(self):
        self.clear_tree()

        # O(N) with 2 base cases: (assume at least 1 period) every round compute two numbers
         and copy an existing list
        self.stock_tree.append([self.initial_stock_value]) # period0
        self.stock_tree.append([self.initial_stock_value * self.d, self.initial_stock_value
         * self.u]) # period1

        for day in range(self.steps - 1):
            self.stock_tree.append([self.stock_tree[-2][0] * (self.d ** 2)] + self.
       stock_tree[-2] + [self.stock_tree[-2][-1] * (self.u ** 2)])

        # Stock tree done, then generate option tree backward induction
        return self.stock_tree

    def generate_option_tree(self):
        import numpy as np
        import copy
        self.option_tree = copy.deepcopy(self.stock_tree)

        last_day_values = []
        for elem in self.option_tree[-1]:
            elem -= self.strike_price
            elem = max(elem, 0)
            last_day_values.append(elem)
        self.option_tree[-1] = last_day_values

        # across days
        for i in range(len(self.option_tree)-2, -1, -1):
            # across values in a day
            for j in range(len(self.option_tree[i])-1, -1, -1):

                deltai = (self.option_tree[i+1][j] - self.option_tree[i+1][j+1]) / (self.
       stock_tree[i+1][j] - self.stock_tree[i+1][j+1])

                xi = vi = math.exp(-self.rate * self.stepsize) * (self.risk_neutral_p *
       self.option_tree[i+1][j+1] + self.risk_neutral_q * self.option_tree[i+1][j])

                self.option_tree[i][j] = vi

        return self.option_tree

    # Need to have had a tree
    def pricing(self):
        return self.option_tree[0][0]

    def simulate_stock(self):
        value = self.initial_stock_value
        path = [value]

        j = 0
        for i in range(self.steps):
            j += 1 if random.random() < self.risk_neutral_p else 0
            value = self.stock_tree[i + 1][j]
            path.append(value)

        days_per_step = self.stepsize * 252
        x = [i * days_per_step for i in range(len(path))]

        plt.plot(x, path)
        plt.xlabel("Days")
        plt.ylabel("Stock Value")
        plt.title(f"Stock Simulation N={self.steps}")
        return path

    def simulate_option(self):
        value = self.option_tree[0][0]
        path = [value]

        j = 0
        for i in range(self.steps):
            j += 1 if random.random() < self.risk_neutral_p else 0
            value = self.option_tree[i + 1][j]
            path.append(value)

        days_per_step = self.stepsize * 252
        x = [i * days_per_step for i in range(len(path))]

        plt.plot(x, path)
        plt.xlabel("Days")
        plt.ylabel("Option Value")
        plt.title(f"Option Simulation N={self.steps}")
        return path

    def plot_stock_tree(self, fname = "binomial_stock_tree.png"):
        for i, level in enumerate(self.stock_tree):
            plt.scatter([i] * len(level), level, color = "C0", s = 12)
        plt.title(f"CRRBinomialStockTreeN{self.steps}")
        plt.xlabel("Days")
        plt.ylabel("Price")
        plt.savefig(fname, bbox_inches = "tight")
        plt.show()

    def plot_option_tree(self, fname = "binomial_stock_tree.png"):
        for i, level in enumerate(self.option_tree):
            plt.scatter([i] * len(level), level, color = "C0", s = 12)
```

```
        plt.title(f"CRRBinomialOptionTreeN{self.steps}")
        plt.xlabel("Days")
        plt.ylabel("Price")
        plt.savefig(fname, bbox_inches = "tight")
        plt.show()

def black_scholes_call(S0, K, r, sigma, T):
    d1 = (math.log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * math.sqrt(T))
    d2 = d1 - sigma * math.sqrt(T)
    return S0 * norm.cdf(d1) - K * math.exp(-r * T) * norm.cdf(d2)

def gbm_euler_maruyama(S0, mu, sigma, T, N, n_paths=1, seed=None):
    # Euler-Maruyama simulation of GBM: S_{n+1} = S_n + mu*S_n*dt + sigma*S_n*dW_n
    rng = np.random.default_rng(seed)
    dt = T / N
    t = np.linspace(0, T, N + 1)

    Z = rng.standard_normal((n_paths, N))
    dW = np.sqrt(dt) * Z

    S = np.empty((n_paths, N + 1))
    S[:, 0] = S0

    for n in range(N):
        S[:, n + 1] = S[:, n] + mu * S[:, n] * dt + sigma * S[:, n] * dW[:, n]

    return t, S

def gbm_exact(S0, mu, sigma, T, N, n_paths=1, seed=None):
    # Exact (log-Euler) simulation of GBM for comparison: S_{n+1} = S_n * exp((mu - 0.5*
        sigma^2)*dt + sigma*sqrt(dt)*Z_n)
    rng = np.random.default_rng(seed)
    dt = T / N
    t = np.linspace(0, T, N + 1)

    Z = rng.standard_normal((n_paths, N))
    increments = (mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z
    log_S = np.log(S0) + np.cumsum(increments, axis=1)

    S = np.empty((n_paths, N + 1))
    S[:, 0] = S0
    S[:, 1:] = np.exp(log_S)

    return t, S

def plotsmart(x=None, ys=(), labels=None, xlabel="", ylabel="", title="", fname=None,
              xscale="linear", single_points=None, kind="line"):
    # To save space, this is an all-in-one helper for every figure, which is written after
        all the plotting commands are written (collecting all needs), and always saves to
        figures/<fname>.png.
    labels = labels or [None] * len(ys)
    if kind == "hist":                      # one histogram panel per dataset
        fig, axs = plt.subplots(1, len(ys), figsize=(5 * len(ys), 4))
        for ax, y, lab in zip(axs, ys, labels):
            y = np.asarray(y)
            if np.unique(y).size <= 2:      # 0/1 indicator (early exercise)
                ax.hist(y, bins=[-0.5, 0.5, 1.5], rwidth=0.6); ax.set_xticks([0, 1])
            else:
                ax.hist(y, bins=40)
            ax.set_xlabel(lab); ax.set_ylabel("count")
        fig.suptitle(title)
    else:
        fig, ax = plt.subplots(figsize=(7, 4.5))
        per_curve_x = isinstance(x, list) and len(x) > 0 and isinstance(x[0], np.ndarray)
        for k, (y, lab) in enumerate(zip(ys, labels)):
            ax.plot(x[k] if per_curve_x else x, y, label=lab)
        for (px, py, lab) in (single_points or []):
            ax.plot([px], [py], "k*", ms=12, label=lab)
        ax.set_xscale(xscale); ax.set_xlabel(xlabel); ax.set_ylabel(ylabel); ax.set_title(
        title)
        if any(labels) or single_points: ax.legend()
        ax.grid(alpha=0.3)
    fig.tight_layout()
    if fname: fig.savefig(f"figures/{fname}.png", dpi=200)
    plt.show()

def perp_params(r, vol, K=K):
    kappa = r / vol**2
    Delta = math.sqrt((kappa - 1/2)**2 + 2*kappa)
    beta = (kappa - 1/2) + Delta          # = 2 r / vol^2
    return beta, K * beta / (beta + 1)

def Vinf(x, r=r, vol=vol, K=K):
    beta, Sstar = perp_params(r, vol, K)
    x = np.asarray(x, float)
    return np.where(x <= Sstar, K - x, K / (beta + 1) * (Sstar / x) ** beta)

class CRRTree(CRRBinomialModel):
    def __init__(self, S0=10, K=10, mu=0.05, vol=0.2, r=0.02, T=1, steps=100, seed=SEED):
        # the base class counts maturity in trading days (T = days/252), so days = round(252
        T)
        super().__init__(initial_stock_value=S0, rate=r, volatility=vol, steps=steps,
                         days=round(252 * T), strike_price=K)
        self.S0, self.K, self.mu, self.vol, self.r = S0, K, mu, vol, r
        self.dt, self.p, self.q = self.stepsize, self.risk_neutral_p, self.risk_neutral_q
        self.rng = np.random.default_rng(seed)
        self.tree = []
        self.build_stock_tree()
        self.decision_bound = 0
        self.hedgings = []

    def build_stock_tree(self):
        return self.generate_stock_tree()

    def S(self, j, i):
        return self.stock_tree[j][i]

    def build_option_tree(self):
        # same backward induction as generate_option_tree, with the early-exercise max (
        American put)
        M, disc = self.steps, math.exp(-self.rate * self.stepsize)
        Sj = [np.array(layer) for layer in self.stock_tree]
        V = [None] * (M + 1); ex = [None] * (M + 1)
        V[M] = np.maximum(self.strike_price - Sj[M], 0); ex[M] = V[M] > 0
        for j in range(M - 1, -1, -1):
            a = np.maximum(self.strike_price - Sj[j], 0)
            changes = self.risk_neutral_p * V[j + 1][1:] + self.risk_neutral_q * V[j +
        1][:-1]
            b = disc * changes
            ex[j] = a > b + 1e-12               # strictly better to exercise (ties continue
        )
            V[j] = np.maximum(a, b)
        self.option_tree, self.exercise = V, ex
        self.exercise_grid = np.zeros((M + 1, M + 1), bool)
        for j in range(M + 1): self.exercise_grid[j, :j + 1] = ex[j]
        # exercise boundary b(t_j): largest stock node in the exercise region (nan if none)
        self.decision_bound = np.array([Sj[j][ex[j]].max() if ex[j].any() else np.nan for j
         in range(M + 1)])
        self.hedgings = hedgingformula(self.option_tree, Sj)
        return self

    def simulate(self):
        # returns the prices and the node index j (= number of up moves) at each step
        ups = np.concatenate(([0], np.cumsum(self.rng.random(self.steps) < self.
        risk_neutral_p)))
        return self.S0 * self.u ** (2 * ups - np.arange(self.steps + 1)), ups     # =
        stock_tree[i][ups[i]]

    def stopping_index(self, ups):
        # optimal stopping: first step where the path is in the exercise region (M = held to
         maturity)
        flags = self.exercise_grid[np.arange(self.steps + 1), ups]
        return int(np.argmax(flags)) if flags.any() else self.steps

def hedgingformula(option_tree, stock_tree):
    # delta at node (j,i) = (V_up - V_down) / (S_up - S_down)   (same ratio as deltai in the
         European model)
    hedgings = []
    for j in range(len(option_tree) - 1):
        hedgings.append((option_tree[j + 1][1:] - option_tree[j + 1][:-1]) /
                        (stock_tree[j + 1][1:] - stock_tree[j + 1][:-1]))
    return hedgings

def CRRDiscretize_run(steps):
    model = CRRTree(steps=steps)
    model.build_option_tree()
    VOs.append(model.option_tree[0][0]); bounds[steps] = model
    return model

def try_paths(model, prompt=("early exercise occurs", "the option is not exercised until
        maturity."), steps=100):
    M = model.steps; p1 = p2 = None
    while p1 is None or p2 is None:
        prices, ups = model.simulate(); tau = model.stopping_index(ups)
        if p1 is None and tau < M and abs(tau * model.dt - 0.5) < 0.1: p1 = (prices, ups,
        tau)
        if p2 is None and tau == M: p2 = (prices, ups, tau)
    return p1, p2

def simulate_10000_paths(vol=0.2, r=0.02, n=10000):
    pnls, stopping_times, early_exercise_probas = [], [], []
    model = CRRTree(vol=vol, r=r, steps=Mbin_sim).build_option_tree()
    V0 = model.option_tree[0][0]                    # fair value under (vol, r)
    for i in range(n):
        simulated_stock_prices, ups = model.simulate()
        tau = model.stopping_index(ups)             # optimal stopping index
        Stau = simulated_stock_prices[tau]
        pnls.append(math.exp(-r * tau * model.dt) * max(K - Stau, 0) - V0)
        stopping_times.append(tau * model.dt)
        early_exercise_probas.append(int(tau < model.steps))
    return np.array(pnls), np.array(stopping_times), np.array(early_exercise_probas)

def simulate_with_priced_boundary(real_vol, n=10000):
    model = CRRTree(vol=real_vol, r=r, steps=Mbin_sim).build_option_tree()   # realised-vol
        world (same dt grid)
    out = {k: [] for k in ["pnl", "tau", "ex", "pnl_opt", "tau_opt"]}
```

```
    for i in range(n):
        prices, ups = model.simulate()
        hit = prices <= exercise_boundary_voldot2 * (1 + 1e-9)   # follow the priced (sigma
        =20%) boundary (tolerance: float noise in the tree nodes)
        tau = int(np.argmax(hit)) if hit.any() else model.steps
        tau_opt = model.stopping_index(ups)                # truly optimal rule under the
        realised vol (same path)
        out["pnl"].append(math.exp(-r * tau * model.dt) * max(K - prices[tau], 0) -
        fair_value_voldot2)
        out["pnl_opt"].append(math.exp(-r * tau_opt * model.dt) * max(K - prices[tau_opt],
        0) - fair_value_voldot2)
        out["tau"].append(tau * model.dt); out["tau_opt"].append(tau_opt * model.dt)
        out["ex"].append(int(tau < model.steps))
    return {k: np.array(v) for k, v in out.items()}
```

Hand-written classes and functions.

```
# Extensions that build on the hand-written code (fast trees, Greeks, PDE solver, LSM, BAW,
      hedging experiments, ...)

def rng_for(k):                                    # independent reproducible numpy stream
      per experiment
    return np.random.default_rng([SEED, k])

def macro(name, text):
    # \newcommand{\name}{text}; the name may contain letters only
    assert name.isalpha(), name
    MACROS[name] = str(text)

def save_fig(fig, name, show=True):
    fig.savefig(f"figures/{name}.pdf")
    if show: plt.show()
    else: plt.close(fig)

def write_table(name, header, rows, align):
    # writes only the tabular environment (booktabs); caption and label live in main.tex
    lines = [r"\begin{tabular}{" + align + "}", r"\toprule", " & ".join(header) + r" \\", r
        "\midrule"]
    lines += [" & ".join(str(c) for c in row) + r" \\" for row in rows]
    lines += [r"\bottomrule", r"\end{tabular}"]
    Path(f"tables/{name}.tex").write_text("\n".join(lines) + "\n")

def timeit(f, *a, **k):
    # one timed call; calls that finish in under 50 ms are repeated five more times and the
        fastest run is reported
    # (the first call of a function pays import and cache overheads that say nothing about
        the method)
    t0 = time.perf_counter(); out = f(*a, **k); secs = time.perf_counter() - t0
    if secs < 0.05:
        for _ in range(5):
            t0 = time.perf_counter(); f(*a, **k); secs = min(secs, time.perf_counter() - t0
        )
    return out, secs

def brownian_grid_paths(N, n_paths, T=1.0, rng=None):
    dW = math.sqrt(T / N) * rng.standard_normal((n_paths, N))
    return np.concatenate([np.zeros((n_paths, 1)), np.cumsum(dW, axis=1)], axis=1)

def scaled_walk_grid_paths(N, n_paths, T=1.0, rng=None):
    xi = rng.choice([-1.0, 1.0], size=(n_paths, N))
    return np.concatenate([np.zeros((n_paths, 1)), math.sqrt(T / N) * np.cumsum(xi, axis=1)
        ], axis=1)

def ks_distance_to_normal(N):
    # sup_x |P(W^(N)_1 <= x) - Phi(x)| : the CDF is a step function, so check both sides of
        every jump
    k = np.arange(N + 1)
    x = (2 * k - N) / np.sqrt(N)
    right = binom.cdf(k, N, 0.5)
    left = np.concatenate([[0.0], right[:-1]])
    return max(np.abs(right - norm.cdf(x)).max(), np.abs(left - norm.cdf(x)).max())

def walk_max_survival(N, a):
    # P(max_{n<=N} W^(N)_{t_n} >= a) for the scaled walk, exact via reflection on the
        lattice
    k = np.ceil(a * math.sqrt(N) - 1e-12).astype(int)           # smallest integer level >=
        a sqrt(N)
    SN = lambda j: binom.sf(np.ceil((N + j) / 2) - 1, N, 0.5)   # P(S_N >= j) with S_N = 2B
        - N
    return SN(k) + binom.sf(np.floor((N + k) / 2), N, 0.5)

def bs_d1d2(S, K, r, sigma, T, q=0.0):
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    return d1, d1 - sigma * np.sqrt(T)

def bs_price(S, K, r, sigma, T, kind="call", q=0.0):
    d1, d2 = bs_d1d2(S, K, r, sigma, T, q)
    if kind == "call":
        return S * np.exp(-q * T) * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) * norm.cdf(-d1)

def bs_greeks(S, K, r, sigma, T, kind="call", q=0.0):
    d1, d2 = bs_d1d2(S, K, r, sigma, T, q)
    pdf, eq, er = norm.pdf(d1), np.exp(-q * T), np.exp(-r * T)
    gamma = eq * pdf / (S * sigma * np.sqrt(T)); vega = S * eq * pdf * np.sqrt(T)
    if kind == "call":
        delta = eq * norm.cdf(d1); rho = K * T * er * norm.cdf(d2)
        theta = -S * eq * pdf * sigma / (2 * np.sqrt(T)) - r * K * er * norm.cdf(d2) + q *
        S * eq * norm.cdf(d1)
    else:
        delta = eq * (norm.cdf(d1) - 1); rho = -K * T * er * norm.cdf(-d2)
        theta = -S * eq * pdf * sigma / (2 * np.sqrt(T)) + r * K * er * norm.cdf(-d2) - q *
         S * eq * norm.cdf(-d1)
    return dict(delta=delta, gamma=gamma, vega=vega, theta=theta, rho=rho)

def implied_vol(price_obs, S, K, r, T, kind="call", q=0.0, lo=1e-4, hi=5.0):
    # Brent root of bs_price(sigma) - price (bs_price is strictly increasing in sigma, so
        the root is unique)
    f = lambda s: bs_price(S, K, r, s, T, kind, q) - price_obs
    if f(lo) > 0 or f(hi) < 0: return np.nan
    return brentq(f, lo, hi, xtol=1e-12)

def _h_pp(z, n):
    # Peizer-Pratt inversion method 2
    return 0.5 + np.sign(z) * np.sqrt(0.25 - 0.25 * np.exp(-(z / (n + 1/3 + 0.1/(n + 1)))
        **2 * (n + 1/6)))

def tree_params(S0, K, r, sigma, T, N, method="crr", q=0.0):
    dt = T / N; a = math.exp((r - q) * dt)
    if method == "crr":
        u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; p = (a - d) / (u - d)
    elif method == "jr":
        u = math.exp((r - q - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt)); d = math.exp((
        r - q - 0.5 * sigma**2) * dt - sigma * math.sqrt(dt)); p = 0.5
    elif method == "tian":
        nu = math.exp(sigma**2 * dt); w = math.sqrt(nu**2 + 2 * nu - 3)
        u = a * nu / 2 * (nu + 1 + w); d = a * nu / 2 * (nu + 1 - w); p = (a - d) / (u - d)
    elif method == "lr":
        assert N % 2 == 1, "Leisen-Reimer needs an odd number of steps"
        d1, d2 = bs_d1d2(S0, K, r, sigma, T, q)
        p = float(_h_pp(d2, N)); ps = float(_h_pp(d1, N)); u = a * ps / p; d = (a - p * u)
        / (1 - p)
    else:
        raise ValueError(method)
    return u, d, p

def tree_price(S0, K, r, sigma, T, N, kind="call", american=False, method="crr", q=0.0,
        smooth=False, nodes=False,
               root_flag=False, ex_every=None):
    # backward induction on the recombining lattice S_{j,i} = S0 u^i d^(j-i); O(N) memory.
    # american: exercise allowed at every layer (or only when j is a multiple of ex_every:
        Bermudan);
    # root_flag: also return whether exercise is strictly optimal at the root (used to find
        b(0;T)).
    u, d, p = tree_params(S0, K, r, sigma, T, N, method, q); dt = T / N; disc = math.exp(-r
         * dt)
    pay = (lambda s: np.maximum(s - K, 0.0)) if kind == "call" else (lambda s: np.maximum(K
         - s, 0.0))
    node = lambda j: S0 * u ** np.arange(j + 1) * d ** (j - np.arange(j + 1))
    if smooth and not american:                          # Broadie-Detemple: last layer = BS
        value over one step
        Sj = node(N - 1); V = bs_price(Sj, K, r, sigma, dt, kind, q); start = N - 2
    else:
        V = pay(node(N)); start = N - 1
    keep, ex0 = {}, False
    for j in range(start, -1, -1):
        cont = disc * (p * V[1:] + (1 - p) * V[:-1])
        if american and (ex_every is None or j % ex_every == 0):
            pj = pay(node(j))
            if j == 0: ex0 = bool(pj[0] > cont[0] + 1e-12)
            V = np.maximum(pj, cont)
        else:
            V = cont
        if nodes and j <= 2: keep[j] = (node(j).copy(), V.copy())
    if nodes: return float(V[0]), keep
    if root_flag: return float(V[0]), ex0
    return float(V[0])

def gbm_euler_terminal(S0, mu, sigma, T, N, M, rng, Z=None):
    # Z (M x N) may be supplied so that the noise is identical to a run of
        gbm_euler_maruyama
    dt = T / N; S = np.full(M, S0, dtype=float)
    for n in range(N):
```

```
        S += mu * S * dt + sigma * S * math.sqrt(dt) * (rng.standard_normal(M) if Z is None
         else Z[:, n])
    return S

def terminal_exact(Z, S0=S0, r=r, sigma=sigma, T=T):
    return S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)

def mc_estimators(M, rng, K=K):
    # returns {method: (estimate, standard error)} for the discounted call payoff, sampling
        cost M payoffs (antithetic: M/2 pairs)
    e = math.exp(-r * T); out = {}
    Z = rng.standard_normal(M); Y = e * np.maximum(terminal_exact(Z) - K, 0)
    out["plain"] = (Y.mean(), Y.std(ddof=1) / math.sqrt(M))
    Zh = Z[: M // 2]; Ya = 0.5 * (e * np.maximum(terminal_exact(Zh) - K, 0) + e * np.
        maximum(terminal_exact(-Zh) - K, 0))
    out["antithetic"] = (Ya.mean(), Ya.std(ddof=1) / math.sqrt(M // 2))
    X = e * terminal_exact(Z); b = np.cov(Y, X)[0, 1] / X.var(ddof=1)            # control
        variate: discounted S_T, E = S0
    Yc = Y - b * (X - S0); out["control variate"] = (Yc.mean(), Yc.std(ddof=1) / math.sqrt(
        M))
    return out

def euler_and_exact_terminal(S0, mu, sigma, T, N, M, rng):
    dt = T / N; Sx = np.full(M, float(S0)); sumZ = np.zeros(M)
    for _ in range(N):
        Z = rng.standard_normal(M); sumZ += Z; Sx *= 1.0 + mu * dt + sigma * math.sqrt(dt)
        * Z
    return Sx, S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * math.sqrt(dt) * sumZ)

def tree_greeks(S0, K, r, sigma, T, N, kind="call", american=False, q=0.0, method="crr"):
    V0, nd = tree_price(S0, K, r, sigma, T, N, kind, american, method, q, nodes=True)
    (S1, V1), (S2, V2) = nd[1], nd[2]; dt = T / N
    d_up = (V2[2] - V2[1]) / (S2[2] - S2[1]); d_dn = (V2[1] - V2[0]) / (S2[1] - S2[0])
    delta = (V1[1] - V1[0]) / (S1[1] - S1[0]); gamma = (d_up - d_dn) / (0.5 * (S2[2] - S2
        [0]))
    theta = (V2[1] - V0) / (2 * dt)
    hv, hr = 1e-4, 1e-5
    vega = (tree_price(S0, K, r, sigma + hv, T, N, kind, american, method, q) - tree_price(
        S0, K, r, sigma - hv, T, N, kind, american, method, q)) / (2 * hv)
    rho = (tree_price(S0, K, r + hr, sigma, T, N, kind, american, method, q) - tree_price(
        S0, K, r - hr, sigma, T, N, kind, american, method, q)) / (2 * hr)
    return dict(price=V0, delta=delta, gamma=gamma, vega=vega, theta=theta, rho=rho)

def mc_greeks(S0, K, r, sigma, T, M, rng):
    Z = rng.standard_normal(M); ST = terminal_exact(Z, S0, r, sigma, T); e = math.exp(-r *
        T); ind = ST > K; out = {}
    se = lambda y: (y.mean(), y.std(ddof=1) / math.sqrt(M))
    out["delta pathwise"] = se(e * ind * ST / S0)
    out["delta LR"] = se(e * np.maximum(ST - K, 0) * Z / (S0 * sigma * math.sqrt(T)))
    h = 0.01 * S0; up = e * np.maximum(terminal_exact(Z, S0 + h, r, sigma, T) - K, 0); dn =
         e * np.maximum(terminal_exact(Z, S0 - h, r, sigma, T) - K, 0)
    out["delta bump (CRN)"] = se((up - dn) / (2 * h))
    out["gamma LR x pathwise"] = se(e * ind * ST / S0**2 * (Z / (sigma * math.sqrt(T)) - 1)
        )
    out["gamma bump (CRN)"] = se((up - 2 * e * np.maximum(ST - K, 0) + dn) / h**2)
    out["vega pathwise"] = se(e * ind * ST * (math.sqrt(T) * Z - sigma * T))
    out["vega LR"] = se(e * np.maximum(ST - K, 0) * ((Z**2 - 1) / sigma - math.sqrt(T) * Z)
        )
    return out

def hedge_experiment(S0, K, r, sig_real, sig_hedge, T, n, M, rng, drift=None):
    drift = r if drift is None else drift; dt = T / n
    Z = rng.standard_normal((M, n)); logS = np.cumsum((drift - 0.5 * sig_real**2) * dt +
        sig_real * math.sqrt(dt) * Z, axis=1)
    Sp = S0 * np.concatenate([np.ones((M, 1)), np.exp(logS)], axis=1)
    tk = np.arange(n + 1) * dt; D = np.exp(-r * tk)
    tau = T - tk[:-1]; d1 = (np.log(Sp[:, :-1] / K) + (r + 0.5 * sig_hedge**2) * tau) / (
        sig_hedge * np.sqrt(tau))
    delta = norm.cdf(d1); gains = (delta * (D[1:] * Sp[:, 1:] - D[:-1] * Sp[:, :-1])).sum(
        axis=1)
    C0 = float(bs_price(S0, K, r, sig_hedge, T)); eps = C0 + gains - D[-1] * np.maximum(Sp
        [:, -1] - K, 0)
    gamma_term = (D[:-1] * norm.pdf(d1) / (sig_hedge * np.sqrt(tau)) * Sp[:, :-1]).sum(axis
        =1) * dt   # integral of D * Gamma * S^2 dt, for the theory line
    return eps, 0.5 * (sig_hedge**2 - sig_real**2) * gamma_term

def pde_put(S0, K, r, sigma, T, Ny=801, Nt=200, american=True, q=0.0, rho=1e7, half_width=
        None):
    L = (5 * sigma * math.sqrt(T) + abs(r - q) * T + 0.5) if half_width is None else
        half_width
    y = math.log(K) + np.linspace(-L, L, Ny); Sg = np.exp(y); dy = y[1] - y[0]; dt = T / Nt
    mu_ = r - q - 0.5 * sigma**2
    a = 0.5 * sigma**2 / dy**2 - mu_ / (2 * dy); b = -sigma**2 / dy**2 - r; c = 0.5 * sigma
        **2 / dy**2 + mu_ / (2 * dy)
    n = Ny - 2; A = diags([a * np.ones(n - 1), b * np.ones(n), c * np.ones(n - 1)], [-1, 0,
         1], format="csc"); I = diags([np.ones(n)], [0], format="csc")
    g = np.maximum(K - Sg, 0.0); gi = g[1:-1]
    left = lambda tau: (K - Sg[0]) if american else (K * math.exp(-r * tau) - Sg[0] * math.
        exp(-q * tau)); right = lambda tau: 0.0
    V = g[1:-1].copy(); tau = 0.0; bnd = [np.nan]; tgrid = [0.0]
    sched = [(dt / 2, 1.0)] * 4 + [(dt, 0.5)] * (Nt - 2)         # Rannacher start, then
        Crank-Nicolson
    for h, th in sched:
        bc_old = np.zeros(n); bc_new = np.zeros(n); bc_old[0] = a * left(tau); bc_old[-1] =
         c * right(tau)
        bc_new[0] = a * left(tau + h); bc_new[-1] = c * right(tau + h)
        rhs = V + (1 - th) * h * (A @ V + bc_old) + th * h * bc_new; M0 = (I - th * h * A).
        tocsc()
        if american:
            act = V <= gi
            for _ in range(40):
                Vn = splu((M0 + diags([rho * act], [0], format="csc")).tocsc()).solve(rhs +
         rho * act * gi)
                new = Vn < gi
                if (new == act).all(): break
                act = new
            V = np.maximum(Vn, gi)
        else:
            V = splu(M0).solve(rhs)
        tau += h
        if abs(tau / dt - round(tau / dt)) < 1e-9 and round(tau / dt) >= 1:
            ex = ((V - gi) < 1e-6) & (gi > 0) if american else np.zeros(n, bool)
            bnd.append(Sg[1:-1][ex].max() if ex.any() else np.nan); tgrid.append(tau)
    Vfull = np.concatenate([[left(T)], V, [0.0]])
    value = float(np.interp(math.log(S0), y, Vfull)) if False else float(__import__("scipy.
        interpolate", fromlist=["CubicSpline"]).CubicSpline(y, Vfull)(math.log(S0)))
    return dict(value=value, S=Sg, V=Vfull, tau=np.array(tgrid), boundary=np.array(bnd))

def lsm_put(S0, K, r, sigma, T, n_dates, M, rng, deg=3, q=0.0):
    dt = T / n_dates; disc = math.exp(-r * dt)
    Z = rng.standard_normal((M, n_dates))
    S = S0 * np.exp(np.cumsum((r - q - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt) * Z,
        axis=1))
    S = np.concatenate([np.full((M, 1), float(S0)), S], axis=1)
    cash = np.maximum(K - S[:, -1], 0.0)
    for j in range(n_dates - 1, 0, -1):
        cash *= disc; pay = np.maximum(K - S[:, j], 0.0); itm = pay > 0
        if itm.sum() > deg + 1:
            coef = np.polyfit(S[itm, j] / K, cash[itm], deg); cont = np.polyval(coef, S[itm
        , j] / K)
            ex = pay[itm] > cont; idx = np.where(itm)[0][ex]; cash[idx] = pay[idx]
    disc_cash = disc * cash; est = max(K - S0, disc_cash.mean())
    return est, disc_cash.std(ddof=1) / math.sqrt(M)

def baw_put(S, K, r, sigma, T, q=0.0):
    b = r - q; Mm = 2 * r / sigma**2; Nn = 2 * b / sigma**2; kap = 1 - math.exp(-r * T)
    q1 = 0.5 * (-(Nn - 1) - math.sqrt((Nn - 1)**2 + 4 * Mm / kap))
    def parts(Ss):
        d1 = (math.log(Ss / K) + (b + 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
        return d1, 1 - math.exp((b - r) * T) * norm.cdf(-d1)
    def g(Ss):
        d1, f = parts(Ss); return K - Ss - float(bs_price(Ss, K, r, sigma, T, "put", q)) +
        f * Ss / q1
    Sc = brentq(g, 1e-6 * K, K)
    d1c, fc = parts(Sc); A1 = -(Sc / q1) * fc
    S = np.asarray(S, float)
    val = np.where(S > Sc, bs_price(S, K, r, sigma, T, "put", q) + A1 * (S / Sc) ** q1, K -
         S)
    return val, Sc

def eep(S_, K_, r_, sig_, T_, N=600):
    # American minus European on the SAME lattice, so the lattice error cancels and the
        premium is exactly 0 when there is no early exercise
    return tree_price(S_, K_, r_, sig_, T_, N, "put", american=True) - tree_price(S_, K_,
        r_, sig_, T_, N, "put", american=False)

class CRRTreeGen(CRRTree):
    # [EXT] American/European call or put with a continuous dividend yield, built on top of
        CRRTree
    def __init__(self, S0=100, K=100, mu=0.05, vol=0.3, r=0.05, T=1, steps=100, kind="call"
        , q=0.0, american=True, seed=SEED):
        super().__init__(S0=S0, K=K, mu=mu, vol=vol, r=r, T=T, steps=steps, seed=seed)
        self.kind, self.div, self.american = kind, q, american
        self.risk_neutral_p = (math.exp((r - q) * self.stepsize) - self.d) / (self.u - self
        .d)
        self.risk_neutral_q = 1 - self.risk_neutral_p; self.p, self.q = self.risk_neutral_p
        , self.risk_neutral_q

    def build_option_tree(self):
        M, disc = self.steps, math.exp(-self.rate * self.stepsize); sgn = 1.0 if self.kind
        == "call" else -1.0
        Sj = [np.array(layer) for layer in self.stock_tree]
        V = [None] * (M + 1); ex = [None] * (M + 1)
        V[M] = np.maximum(sgn * (Sj[M] - self.strike_price), 0); ex[M] = V[M] > 0
        for j in range(M - 1, -1, -1):
            a = np.maximum(sgn * (Sj[j] - self.strike_price), 0)
            b = disc * (self.risk_neutral_p * V[j + 1][1:] + self.risk_neutral_q * V[j +
        1][:-1])
            ex[j] = (a > b + 1e-12) if self.american else np.zeros(len(a), bool)
            V[j] = np.maximum(a, b) if self.american else b
        self.option_tree, self.exercise = V, ex
        self.exercise_grid = np.zeros((M + 1, M + 1), bool)
        for j in range(M + 1): self.exercise_grid[j, :j + 1] = ex[j]
        pick = np.min if self.kind == "call" else np.max
        self.decision_bound = np.array([pick(Sj[j][ex[j]]) if ex[j].any() else np.nan for j
         in range(M + 1)])
```

```
        self.hedgings = hedgingformula(self.option_tree, Sj)
        return self

def pnl_paths(model, n, p_up, rng, V0=None, r_=None):
    Mm = model.steps; r_ = model.r if r_ is None else r_; V0 = model.option_tree[0][0] if
        V0 is None else V0
    ups = np.concatenate([np.zeros((n, 1), np.int32), np.cumsum(rng.random((n, Mm)) < p_up,
         axis=1, dtype=np.int32)], axis=1)
    flags = model.exercise_grid[np.arange(Mm + 1)[None, :], ups]
    any_ = flags.any(axis=1); tau = np.where(any_, flags.argmax(axis=1), Mm)
    Stau = model.S0 * model.u ** (2 * ups[np.arange(n), tau] - tau)
    return np.exp(-r_ * tau * model.dt) * np.maximum(model.K - Stau, 0) - V0, tau * model.
        dt, tau < Mm

def risk_metrics(x, V0):
    # the buyer's loss is capped at the premium, so the left tail is uninformative (more
        than half the paths lose all of it);
    # the tail that matters is the SELLER's: seller loss = buyer P&L. Report P(total loss),
        sd and the right tail of the buyer's P&L.
    q95, q99 = np.quantile(x, 0.95), np.quantile(x, 0.99)
    return dict(mean=x.mean(), sd=x.std(), ptot=(x < -V0 + 1e-9).mean(), q95=q95, q99=q99,
        es95=x[x >= q95].mean())

def b0_bisect(T_, M_, lo=3.0, hi=10.0, tol=1e-4):
    # largest spot at which exercise is optimal at date 0 (exercise region = {x <= b}); the
        root decision is monotone in x
    while hi - lo > tol:
        mid = 0.5 * (lo + hi)
        if tree_price(mid, 10, 0.02, 0.2, T_, M_, "put", american=True, root_flag=True)[1]:
         lo = mid
        else: hi = mid
    return 0.5 * (lo + hi)
```

```
def add_row(name, fn, key=None):
    (val, secs) = timeit(fn); val, se = (val if isinstance(val, tuple) else (val, None))
    rows.append([name, f"{val:.4f}", f"{val-BS:+.4f}", "--" if se is None else f"{se:.4f}",
        f"{1000*secs:.1f}"])
    if key: dc[key] = (val, se, secs)
    return val, secs

def first_hit(fn, Ns_list):
    for N in Ns_list:
        v, secs = timeit(fn, N)
        if abs(v - BS) <= tol: return N, secs
    return None, float("nan")

def add_am(name, fn, key=None):
    (val, secs) = timeit(fn); val, se = (val if isinstance(val, tuple) else (val, None))
    rows.append([name, f"{val:.5f}", f"{val-V_ref:+.5f}", "--" if se is None else f"{se:.4f
        }", f"{1000*secs:.0f}"])
    if key: da[key] = (val, se, secs)
    return val

def fix_sign(v):
    v = str(v)
    m_ = re.fullmatch(r"([-+]?)(\d+(?:\.\d+)?)e([-+]0*(\d+)", v)          # 8.4e-02 -> 8.4\
        times10^{-2}
    if m_:
        sgn, mant, es, ex = m_.groups(); return (("\\ensuremath{-}" if sgn == "-" else "")
        + r"\ensuremath{" + mant + r"\times10^{" + ("-" if es == "-" else "") + ex + "}}")
    return ("\\ensuremath{-}" + v[1:]) if v.startswith("-") else (("\\ensuremath{+}" + v
        [1:]) if v.startswith("+") else v)
```

Extensions.

References

Giovanni Barone-Adesi and Robert E. Whaley. Efficient analytic approximation of American option values. *Journal of Finance*, 42(2):301–320, 1987.

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

Mark Broadie and Jerome Detemple. American option valuation: new bounds, approximations, and a comparison of existing methods. *Review of Financial Studies*, 9(4):1211–1250, 1996.

Mark Broadie, Paul Glasserman, and Steven Kou. A continuity correction for discrete barrier options. *Mathematical Finance*, 7(4):325–349, 1997.

John C. Cox, Stephen A. Ross, and Mark Rubinstein. Option pricing: a simplified approach. *Journal of Financial Economics*, 7(3):229–263, 1979.

Francine Diener and Marc Diener. Asymptotics of the price oscillations of a European call option in a tree model. *Mathematical Finance*, 14(2):271–293, 2004.

Monroe D. Donsker. An invariance principle for certain probability limit theorems. *Memoirs of the American Mathematical Society*, 6, 1951.

Peter A. Forsyth and Kenneth R. Vetzal. Quadratic convergence for valuing American options using a penalty method. *SIAM Journal on Scientific Computing*, 23(6):2095–2122, 2002.

Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, 2004.

Ioannis Karatzas and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*, volume 113 of *Graduate Texts in Mathematics*. Springer, 1991.

Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, 1992.

Dietmar P. J. Leisen and Matthias Reimer. Binomial models for option valuation: examining and improving convergence. *Applied Mathematical Finance*, 3(4):319–346, 1996.

Francis A. Longstaff and Eduardo S. Schwartz. Valuing American options by simulation: a simple least-squares approach. *Review of Financial Studies*, 14(1):113–147, 2001.

Robert C. Merton. Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

Goran Peskir and Albert Shiryaev. *Optimal Stopping and Free-Boundary Problems*. Birkhäuser, 2006.

Rolf Rannacher. Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43:309–327, 1984.

Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.

Yisong Tian. A modified lattice approach to option pricing. *Journal of Futures Markets*, 13(5): 563–577, 1993.